

Fundamental Java and Spring Boot








Somkiat

Home








Update Info 1
View Activity Log 10+
...

Timeline
About
Friends 3,138
Photos
More ▾


When did you work at Opendream?
×

...
22 Pending Items


Intro

Software Craftsmanship


Software Practitioner at สยามชำนาญกิจ พ.ศ. 2556


Agile Practitioner and Technical at SPRINT3r


Post

Photo/Video

Live Video

Life Event


What's on your mind?


Public ▾

Post



Somkiat Puisungnoen
15 mins · Bangkok · ▾

Java and Bigdata

...



Facebook interface for the page **somkiat.cc**. The top navigation bar includes the Facebook logo, a search bar with "somkiat.cc", and user profile icons for "Somkiat" and "Home". The main navigation bar shows "Page" (selected), "Messages", "Notifications" (3), "Insights", "Publishing Tools", "Settings", and "Help".

The page content area features a large video player showing a man in a white Superman t-shirt with "SOMKIAT.CC" on it, posing against a white wall. A blue call-to-action button is overlaid on the video: "Help people take action on this Page. x". Below the video, there are buttons for "Liked", "Following", "Share", and "...". A blue button labeled "+ Add a Button" is also visible.

The left sidebar shows the page name "somkiat.cc" and handle "@somkiat.cc". Below this is a menu with "Home" (selected), "Posts", "Videos", and "Photos".



**[https://github.com/up1/
course-java-spring-2026](https://github.com/up1/course-java-spring-2026)**



Develop RESTFuI API

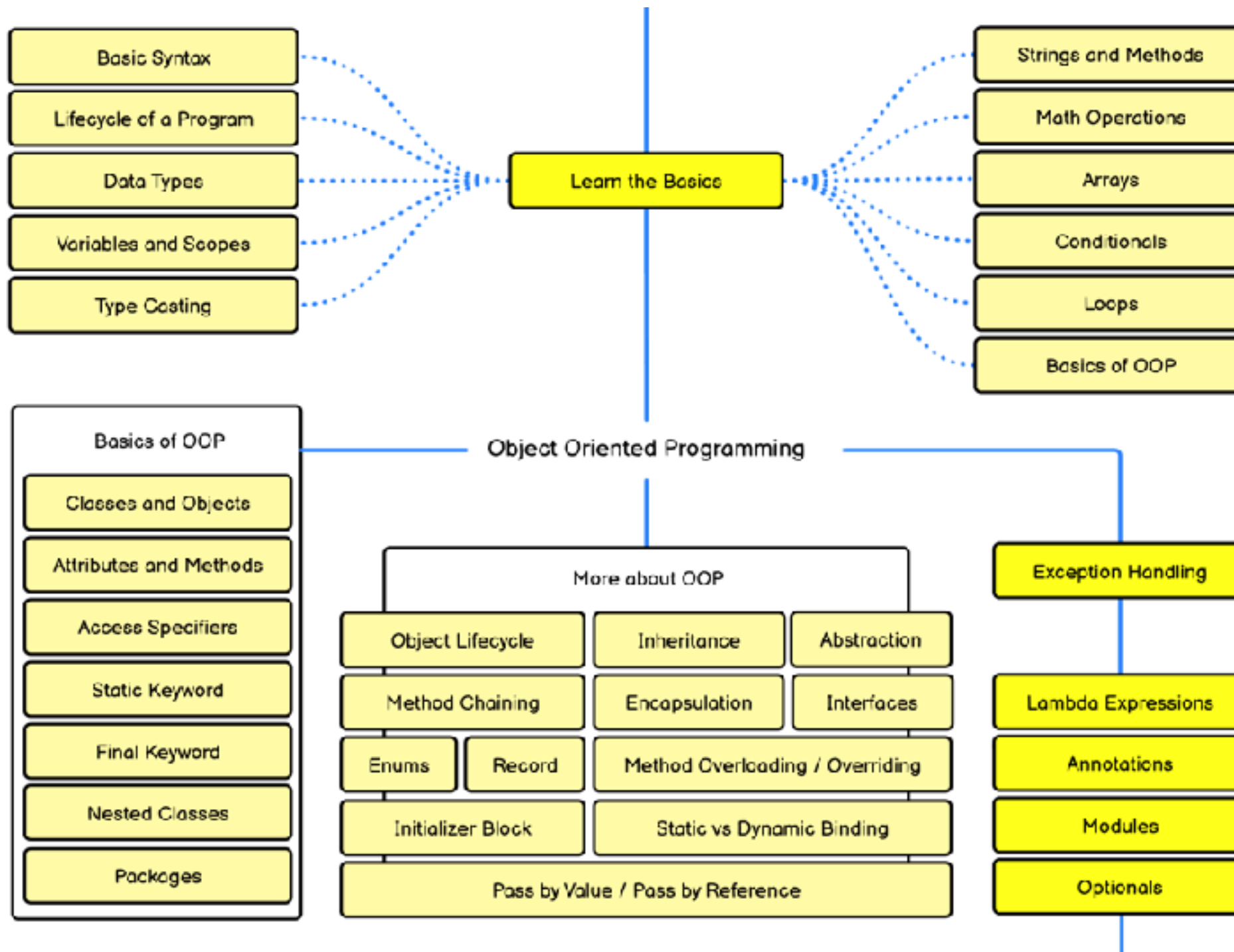


Leaning Java

Modern Java (generic, optional and lambda)
Error handling
Object-Oriented Concepts with Java
Data structure with Java collection framework
Modern Java (generic, optional and lambda)
Testing with JUnit and Mockito



Java Developer Roadmap



<https://roadmap.sh/java>



Basic of REST

REST (REpresentational State Transfer)

6 REST constraints

REST vs RESTful API

Design principles



Spring Boot

Goals of Spring Boot

Better project structure of Spring Boot

Develop RESTful APIs

Error handling

Working with database with Spring Data

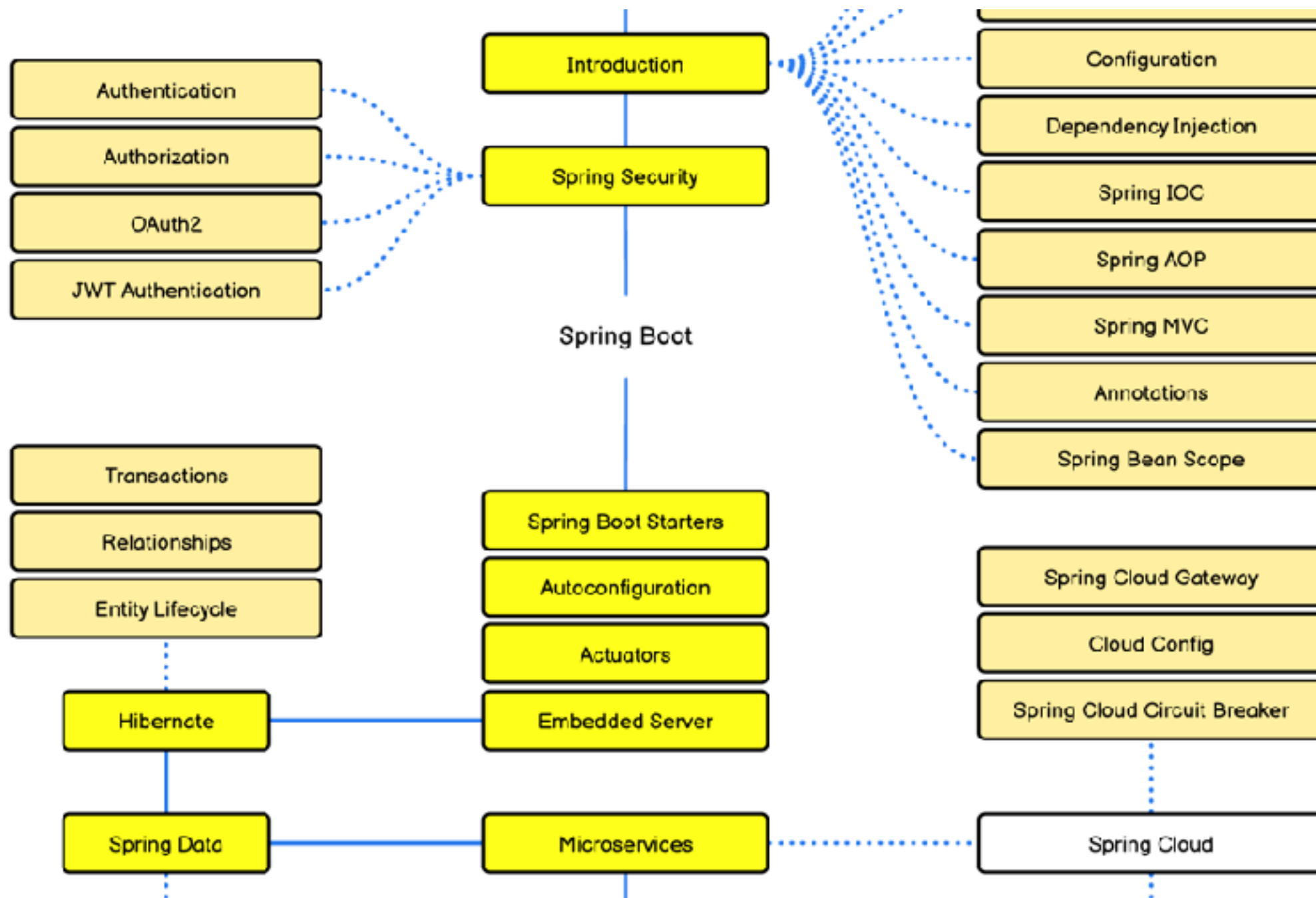
Manage connections and transactions

Testing

Observable service (log, trace, metric)



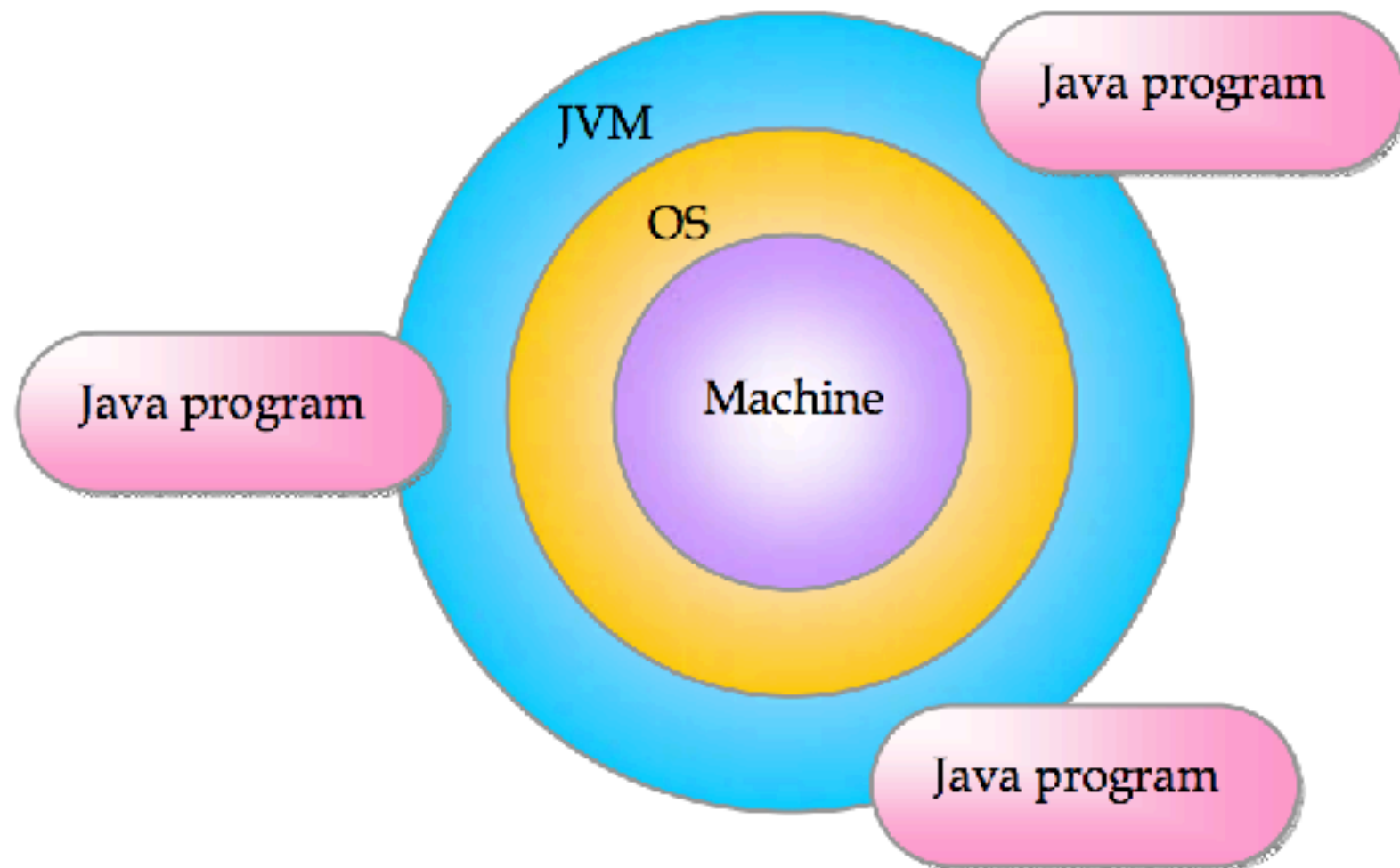
Spring Boot Roadmap



<https://roadmap.sh/spring-boot>



Write Once Run Anywhere ?



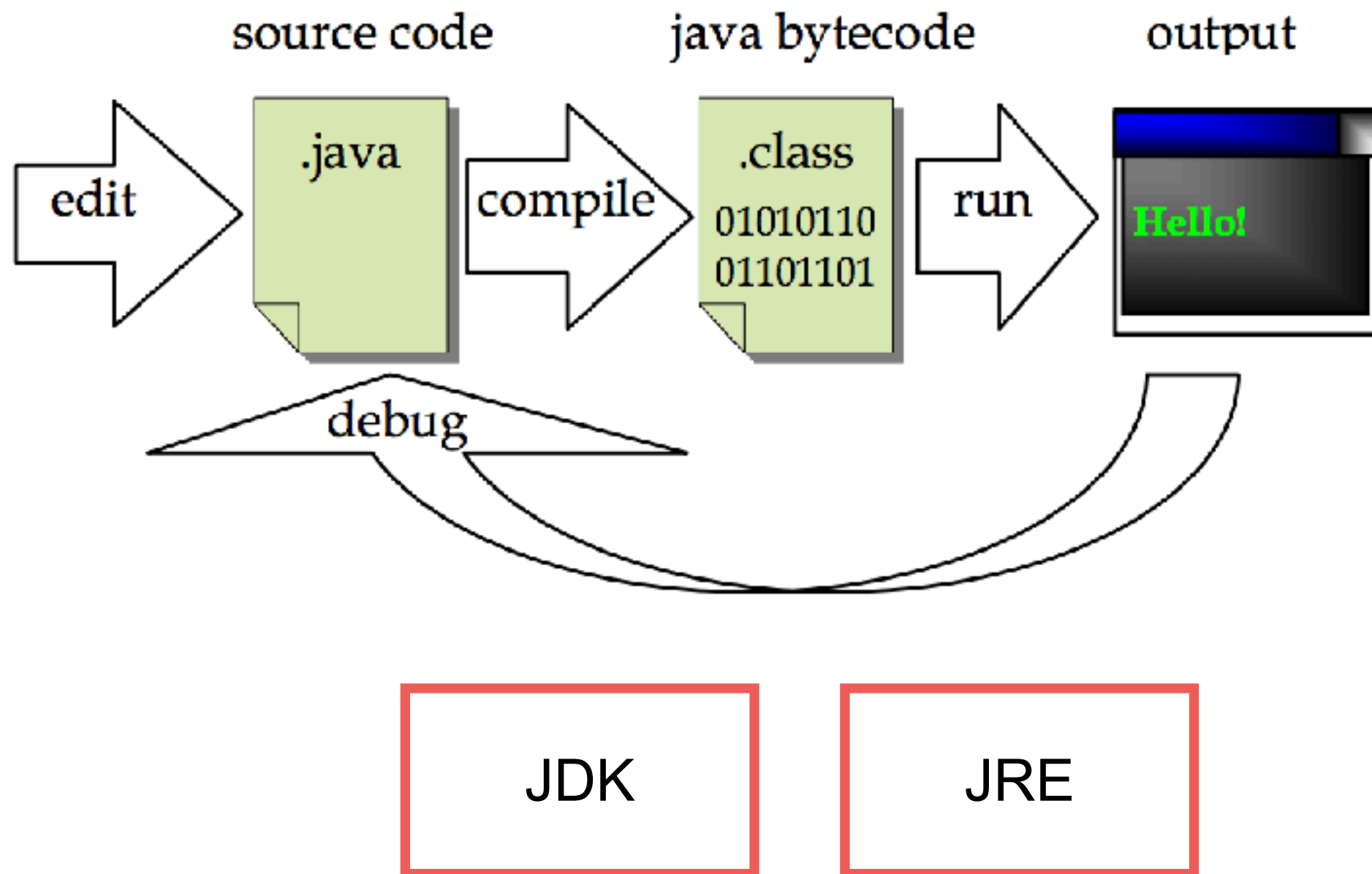
Structure of Java program

Create file HelloWorld.java

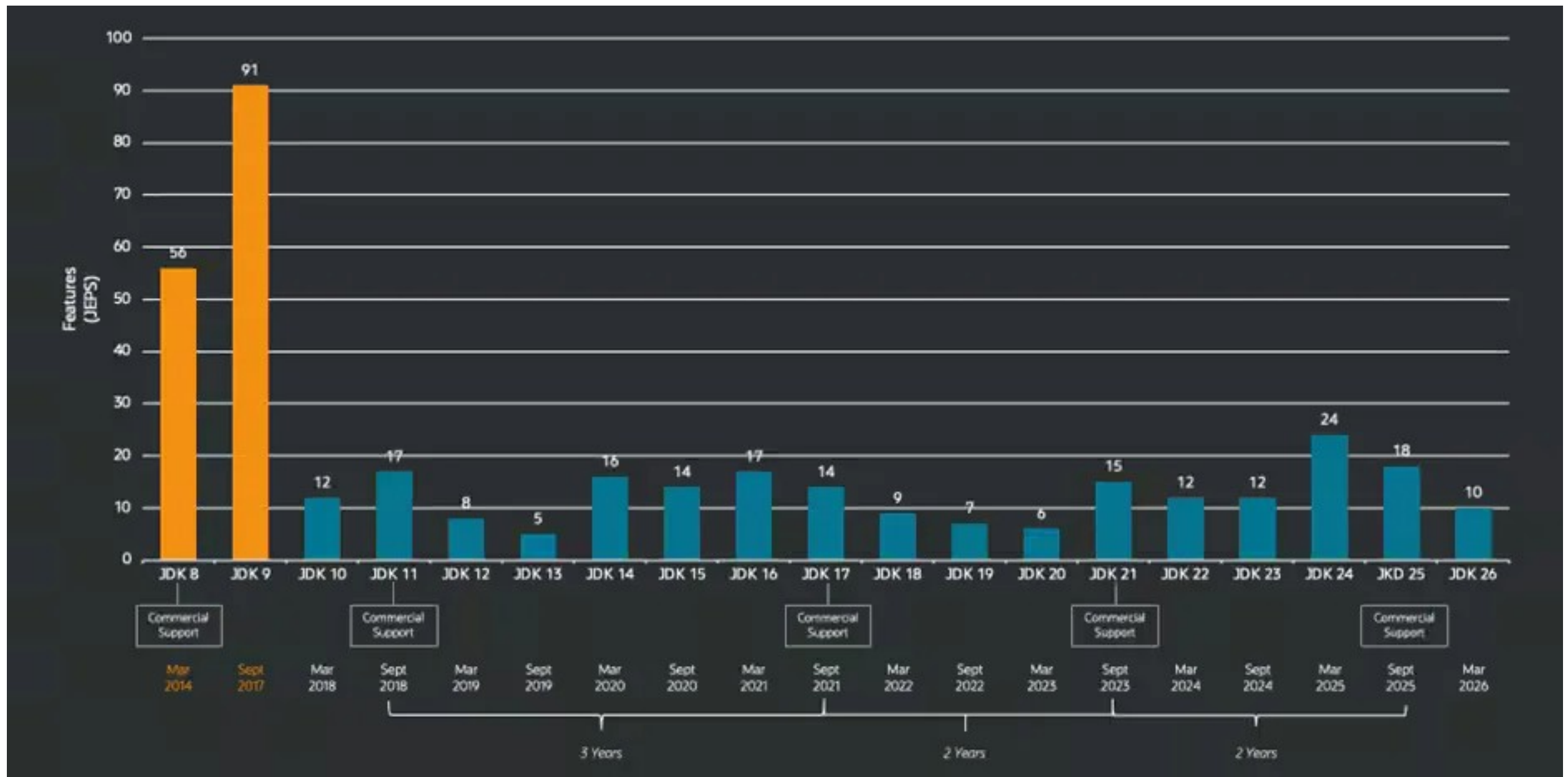
```
1 public class HelloWorld {  
2  
3     // Show message in a console  
4     public static void main(String[] args) {  
5         System.out.println("Hello World");  
6  
7     }  
8  
9 }
```



Java Development Workflow



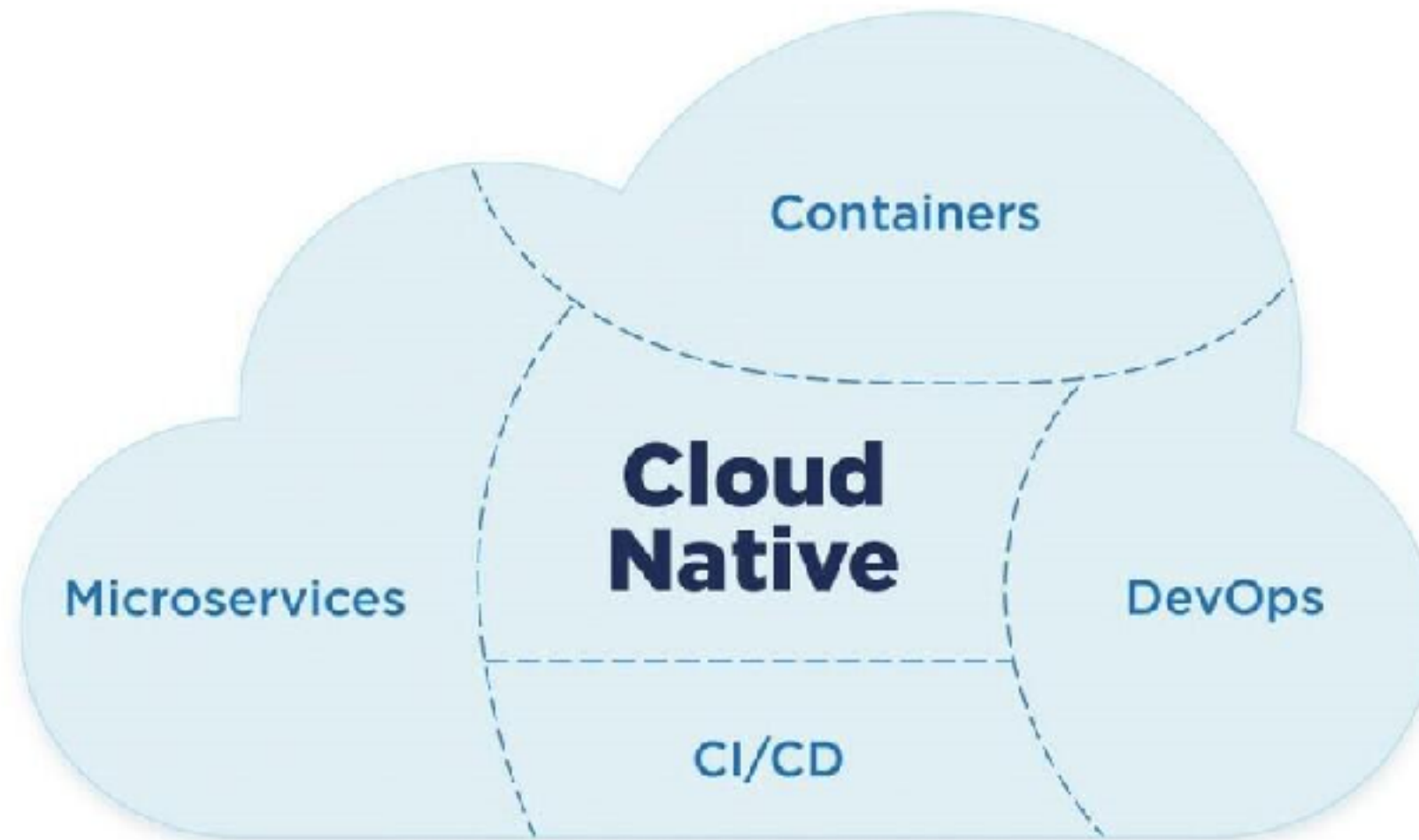
History of Java from 8 to 26



<https://www.infoq.com/news/2026/03/java26-released/>



Cloud Native Architecture



Java in Cloud Native App

Improve performance and resource usages

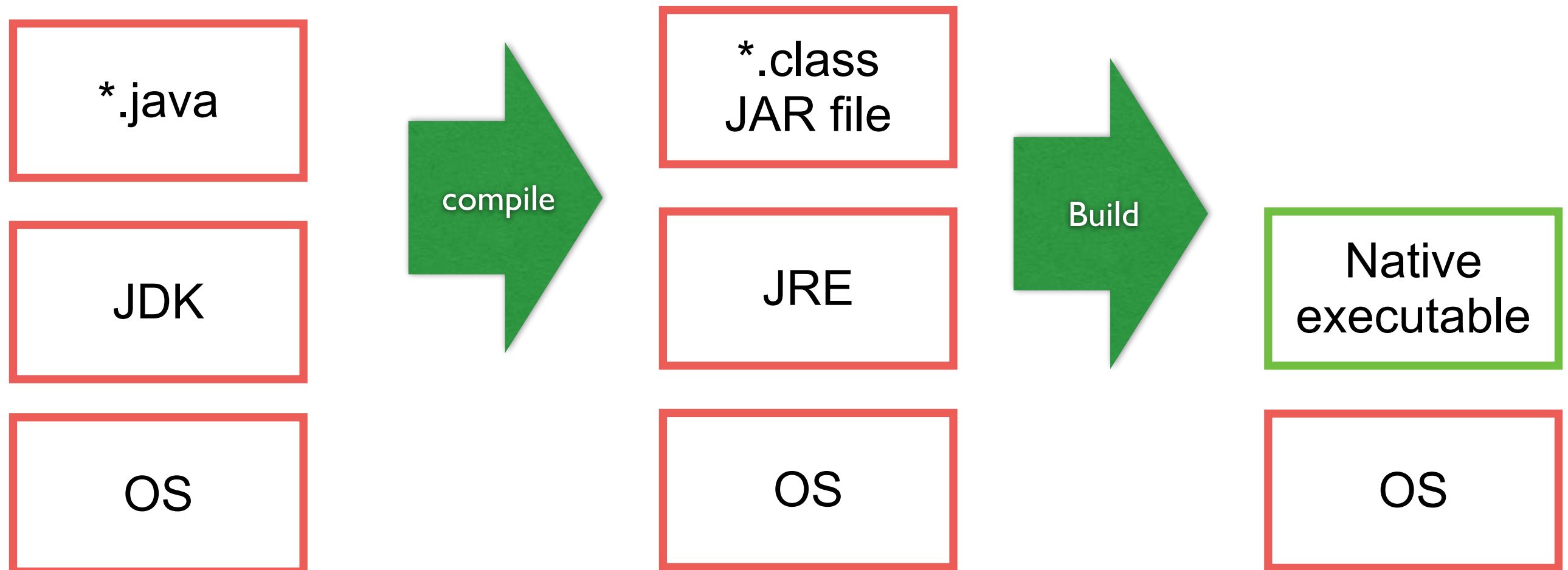


QUARKUS

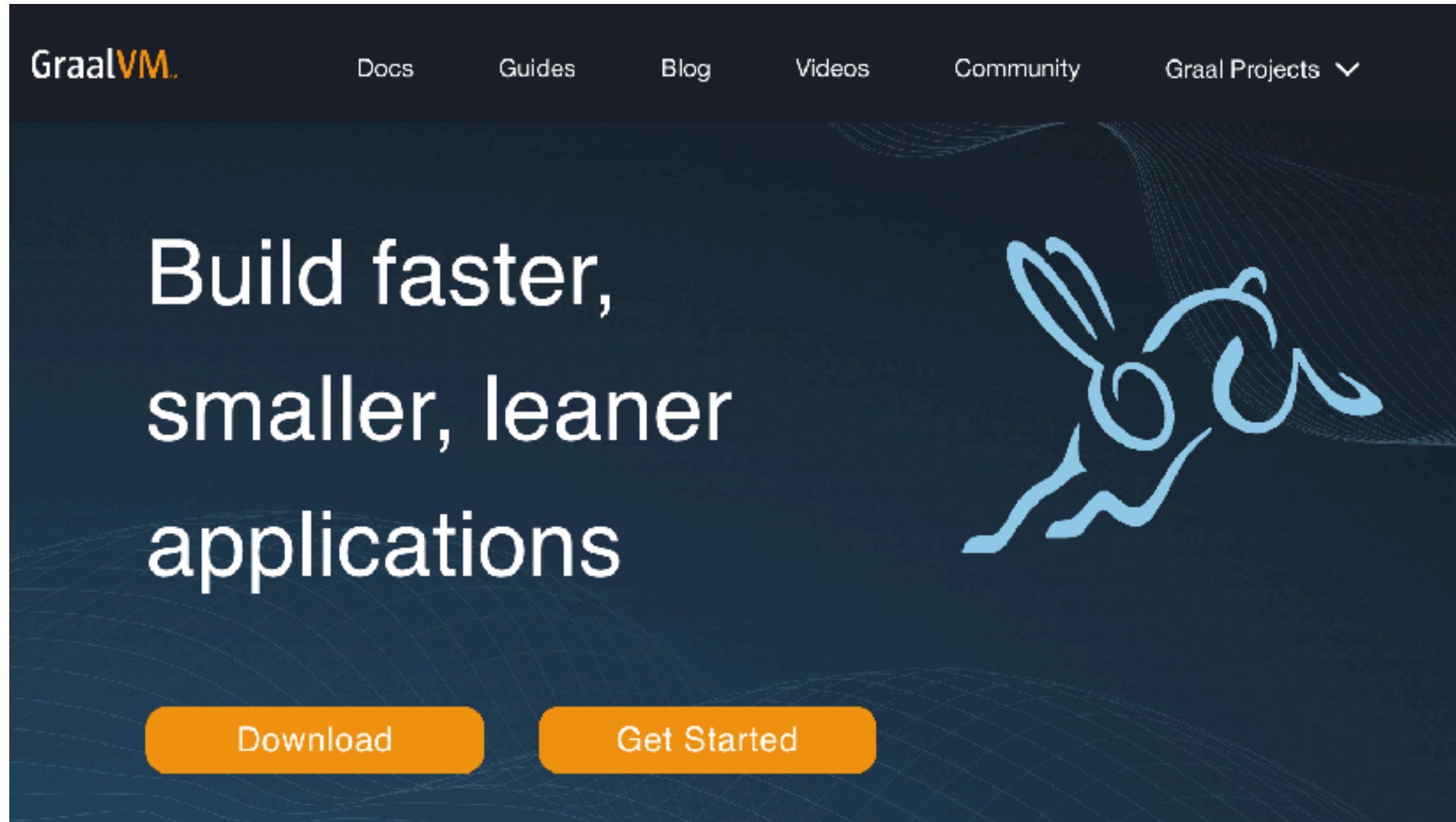
GraalVM™



Java in Cloud Native



Working with GraalVM



<https://www.graalvm.org/>



Native Image of GraalVM

Use less resources (compare with JRE)

Start in milliseconds

Deliver peak performance immediately (no warmup)

Lightweight package

Reduce attack surface

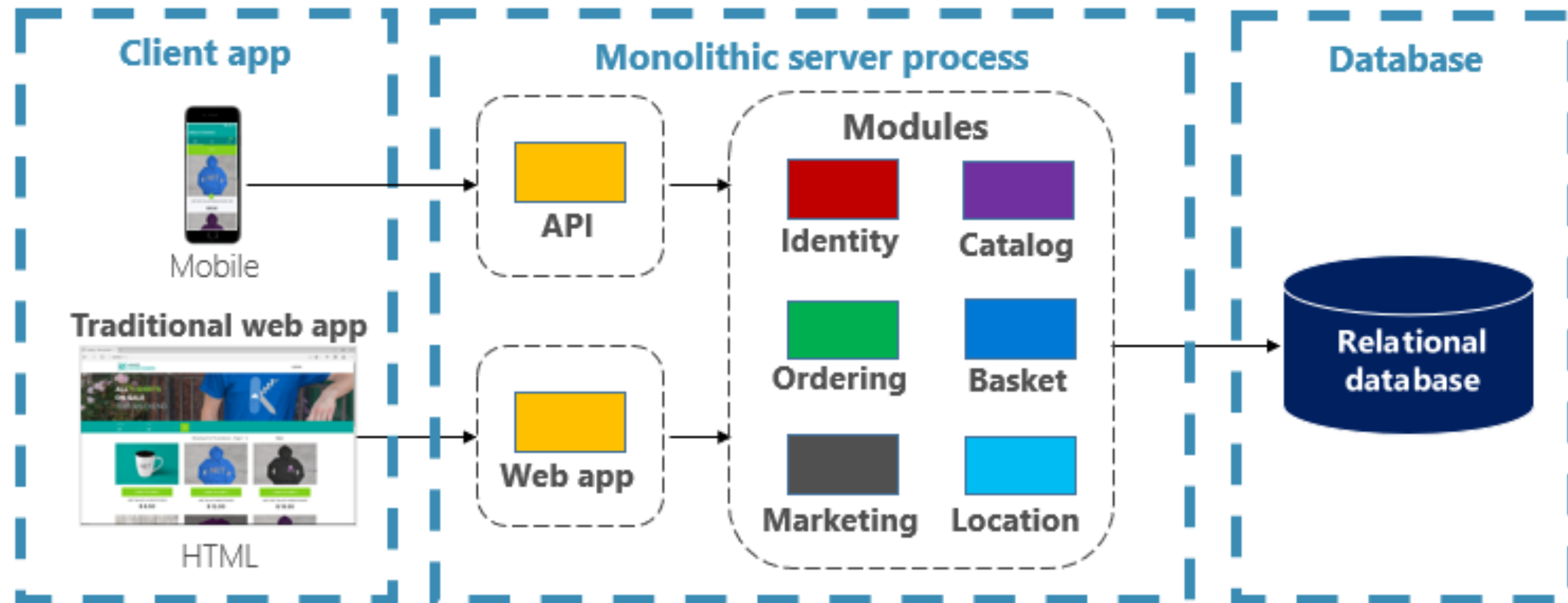
<https://www.graalvm.org/latest/reference-manual/native-image/>



Cloud Native Architecture Design



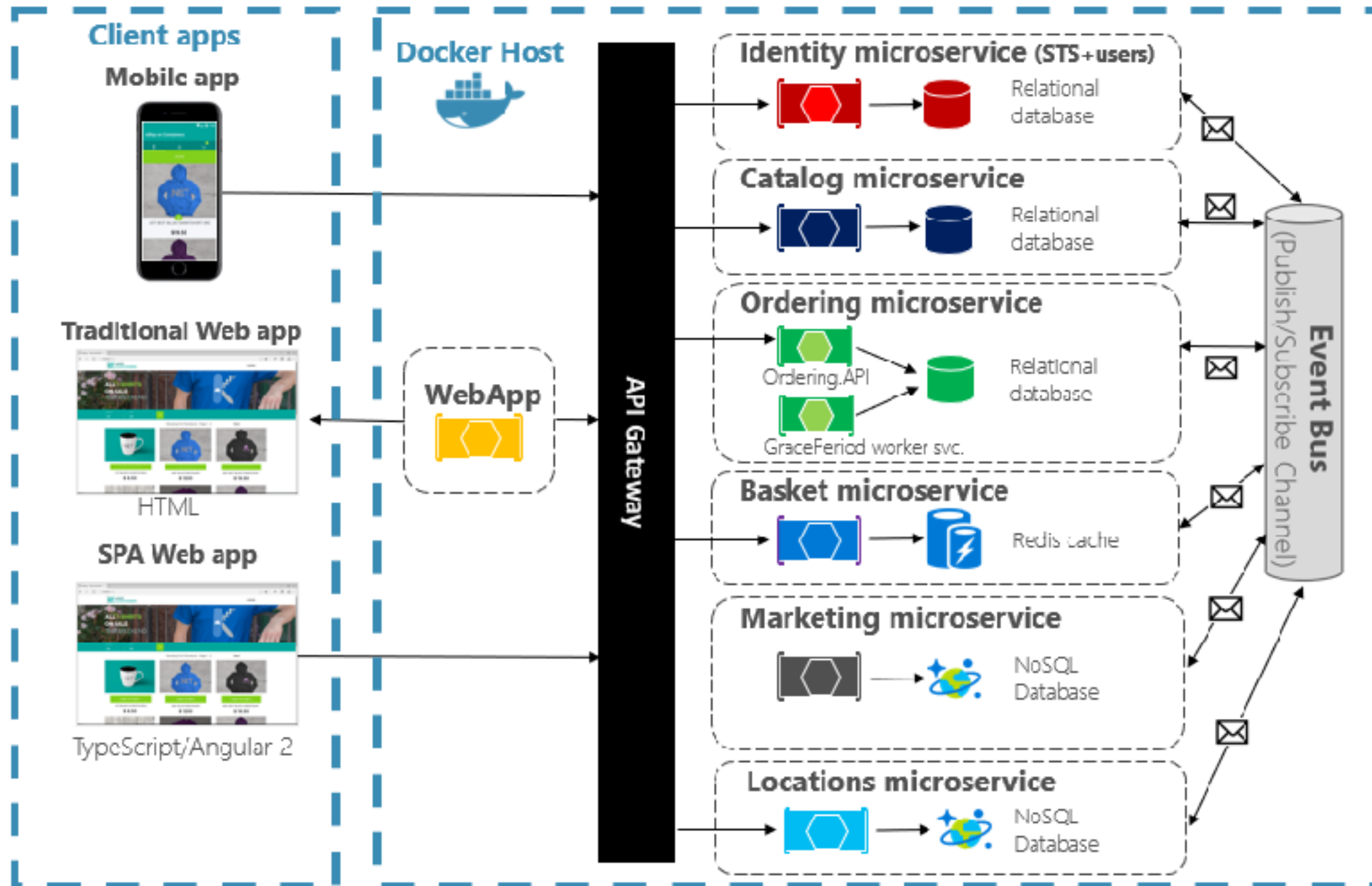
Monolithic



<https://learn.microsoft.com/th-th/dotnet/architecture/cloud-native/introduction>



Microservices



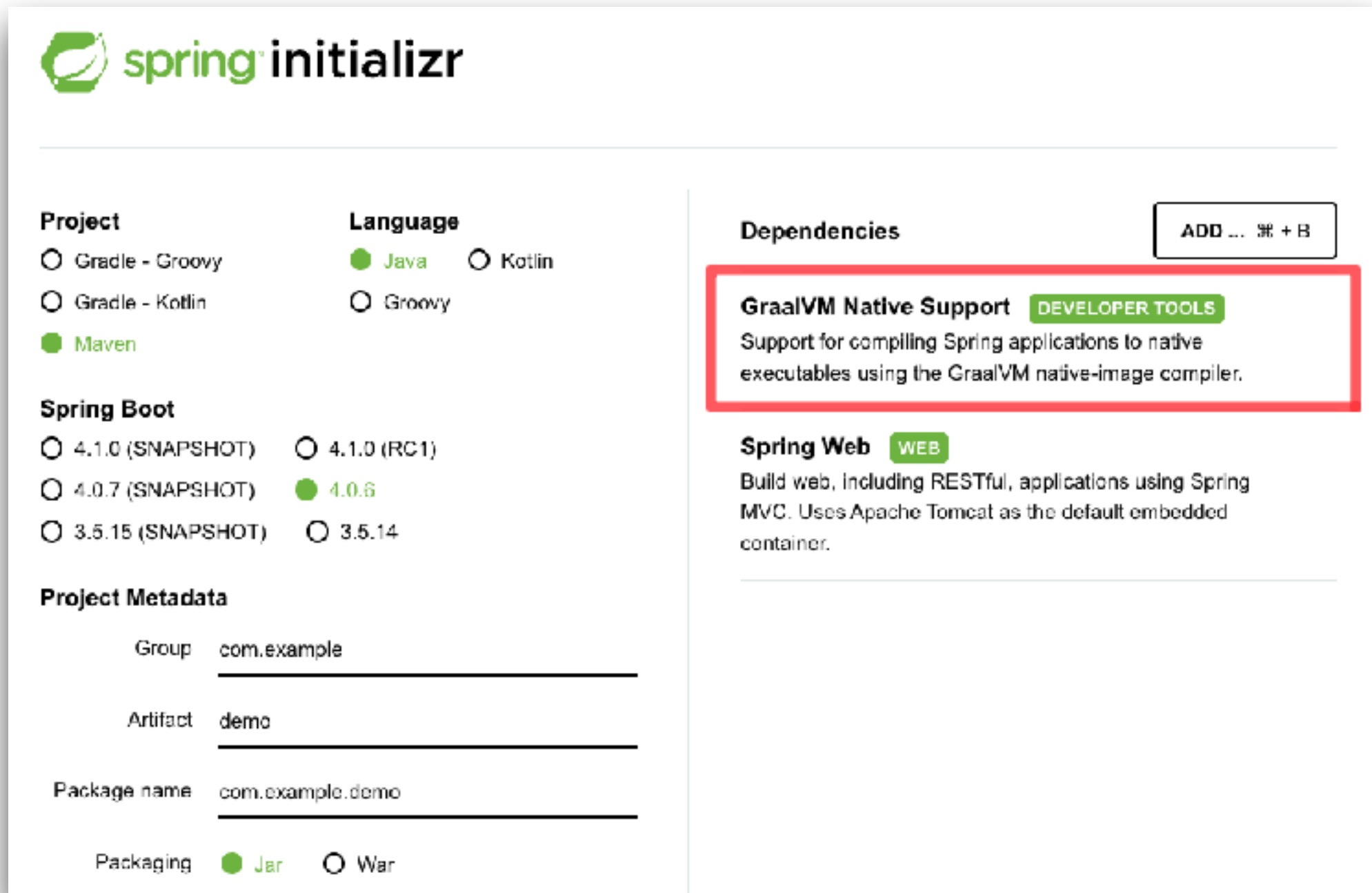
<https://learn.microsoft.com/th-th/dotnet/architecture/cloud-native/introduction>



GraalVM in Spring Boot



GraalVM in Spring Boot



The image shows the Spring Initializr web form. It is divided into several sections: Project, Language, Spring Boot, Project Metadata, Dependencies, and a right-hand sidebar. The 'Project' section has radio buttons for 'Gradle - Groovy', 'Gradle - Kotlin', and 'Maven' (selected). The 'Language' section has radio buttons for 'Java' (selected), 'Kotlin', and 'Groovy'. The 'Spring Boot' section has radio buttons for '4.1.0 (SNAPSHOT)', '4.1.0 (RC1)', '4.0.7 (SNAPSHOT)', '4.0.6' (selected), '3.5.15 (SNAPSHOT)', and '3.5.14'. The 'Project Metadata' section has text input fields for 'Group' (com.example), 'Artifact' (demo), and 'Package name' (com.example.demo). The 'Packaging' section has radio buttons for 'Jar' (selected) and 'War'. The 'Dependencies' section has a red box highlighting 'GraalVM Native Support' with a 'DEVELOPER TOOLS' tag and a description: 'Support for compiling Spring applications to native executables using the GraalVM native-image compiler.' Below this is 'Spring Web' with a 'WEB' tag and a description: 'Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.' There is an 'ADD ...' button with a plus icon in the top right of the Dependencies section.

Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☒ Maven

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 4.1.0 (SNAPSHOT)

☐ 4.1.0 (RC1)

☐ 4.0.7 (SNAPSHOT)

☒ 4.0.6

☐ 3.5.15 (SNAPSHOT)

☐ 3.5.14

Project Metadata

Group

Artifact

Package name

Packaging ☒ Jar ☐ War

Dependencies ADD ... + B

GraalVM Native Support DEVELOPER TOOLS

Support for compiling Spring applications to native executables using the GraalVM native-image compiler.

Spring Web WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

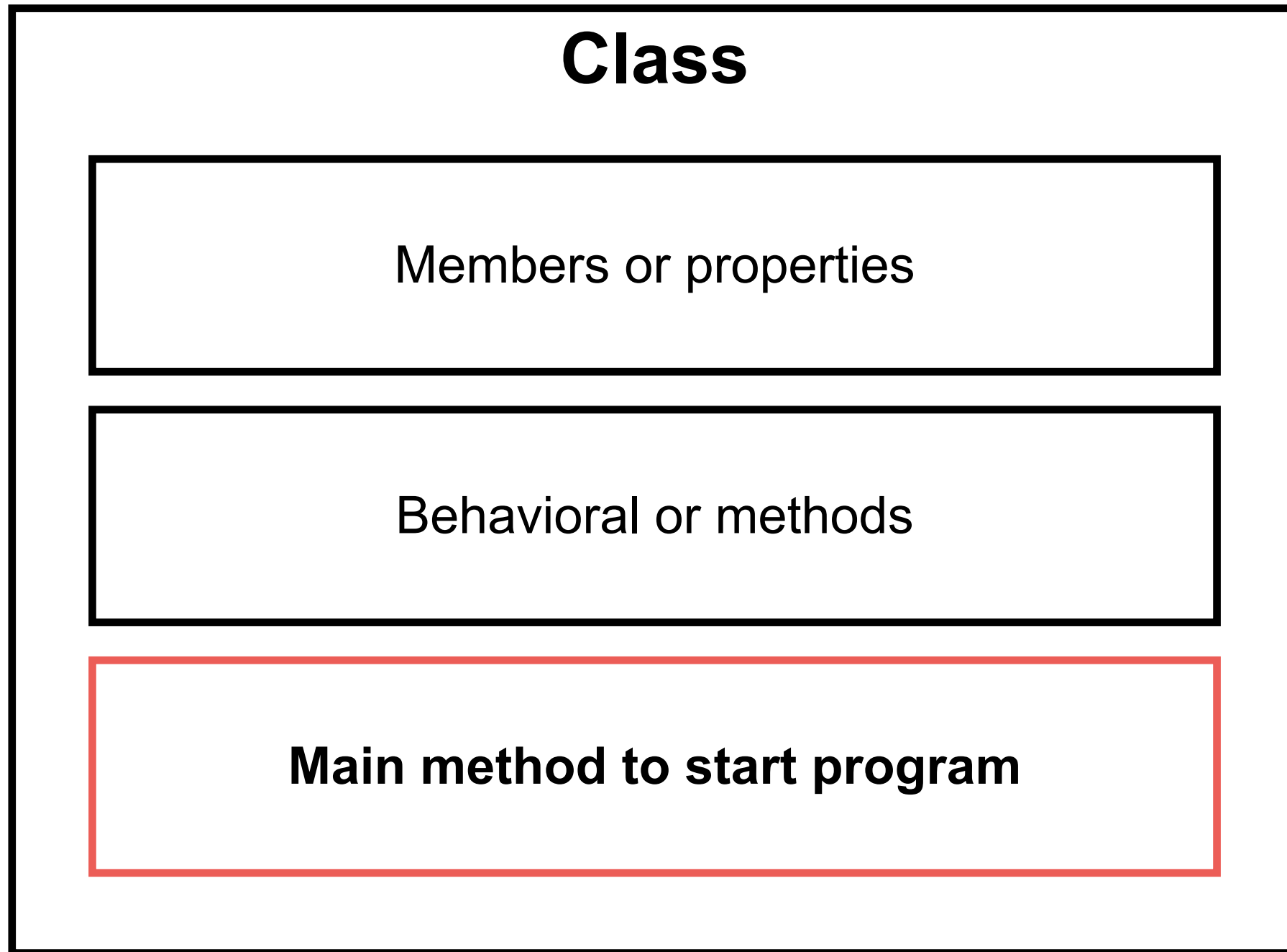
<https://start.spring.io/>



Object-Oriented Programming



Structure of Java program



Thinking before create class



Thinking before create class



Single Responsibility Principle



Single Responsibility Principle

Just because you *can* doesn't mean you *should*.



Open-Closed Principle



Open-Closed Principle

Open-chest surgery isn't needed when putting on a coat.

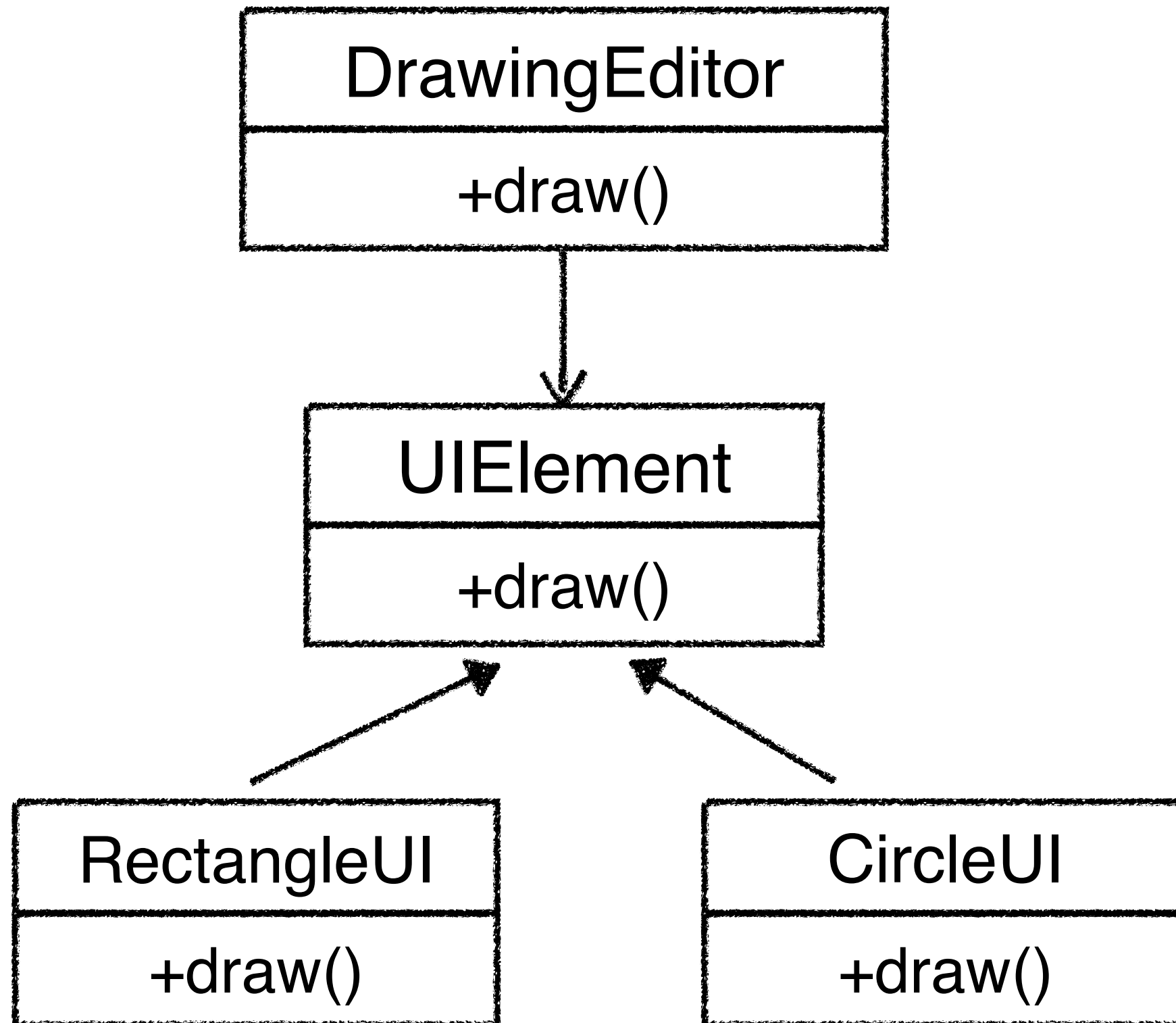


Example

DrawingEditor

+draw()
-drawCircle()
-drawRectangle()





Workshop

OOP with Harry Potter

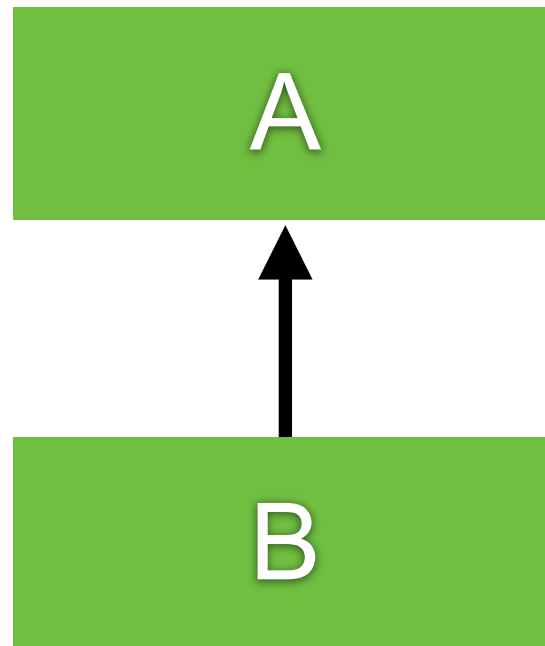
<https://codingdojo.org/kata/Potter/>



Inheritance

Ability to derive a new class from existing class

superclass / base class



subclass / derived class

Additional properties and behaviors



How to create ?

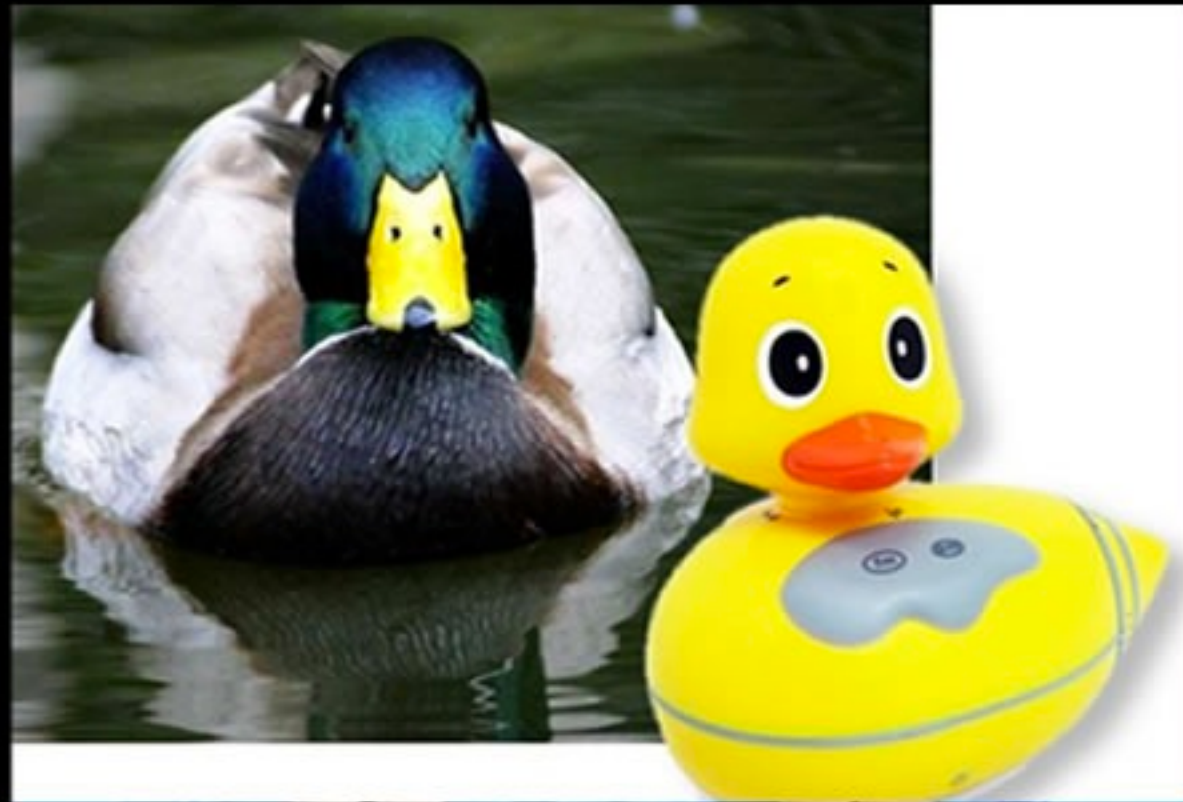
Extends from base class only one class

```
class BaseClass {  
  
}
```

```
class SubClass extends BaseClass {  
  
}
```



Liskov Substitution Principle



Liskov Substitution Principle

If it looks like a duck and quacks like a duck but needs batteries, you probably have the wrong abstraction.



OOP with Java Part 2

Abstract classes
Interfaces



Abstract classes

Placeholder for a method that fully defined in a subclass

Class have abstract modifier

Method can have abstract modifier

Abstract method can't be private

Abstract method can't have body

*“Class that contains an abstract method is called
Abstract class”*



Classes

*“Class that contains an abstract method is called
Abstract class”*

*“Class that not contains an abstract method is called
Concrete class”*



Abstract classes

```
abstract class Employee {
```

```
    private int id;
```

```
    public abstract double getSalary();
```

```
    public boolean sameSalary(Employee other) {  
        return this.getSalary() == other.getSalary();  
    }
```

```
}
```



Pitfall of abstract class

You can't create instances of an abstract class
Abstract class can only use to derive more classes



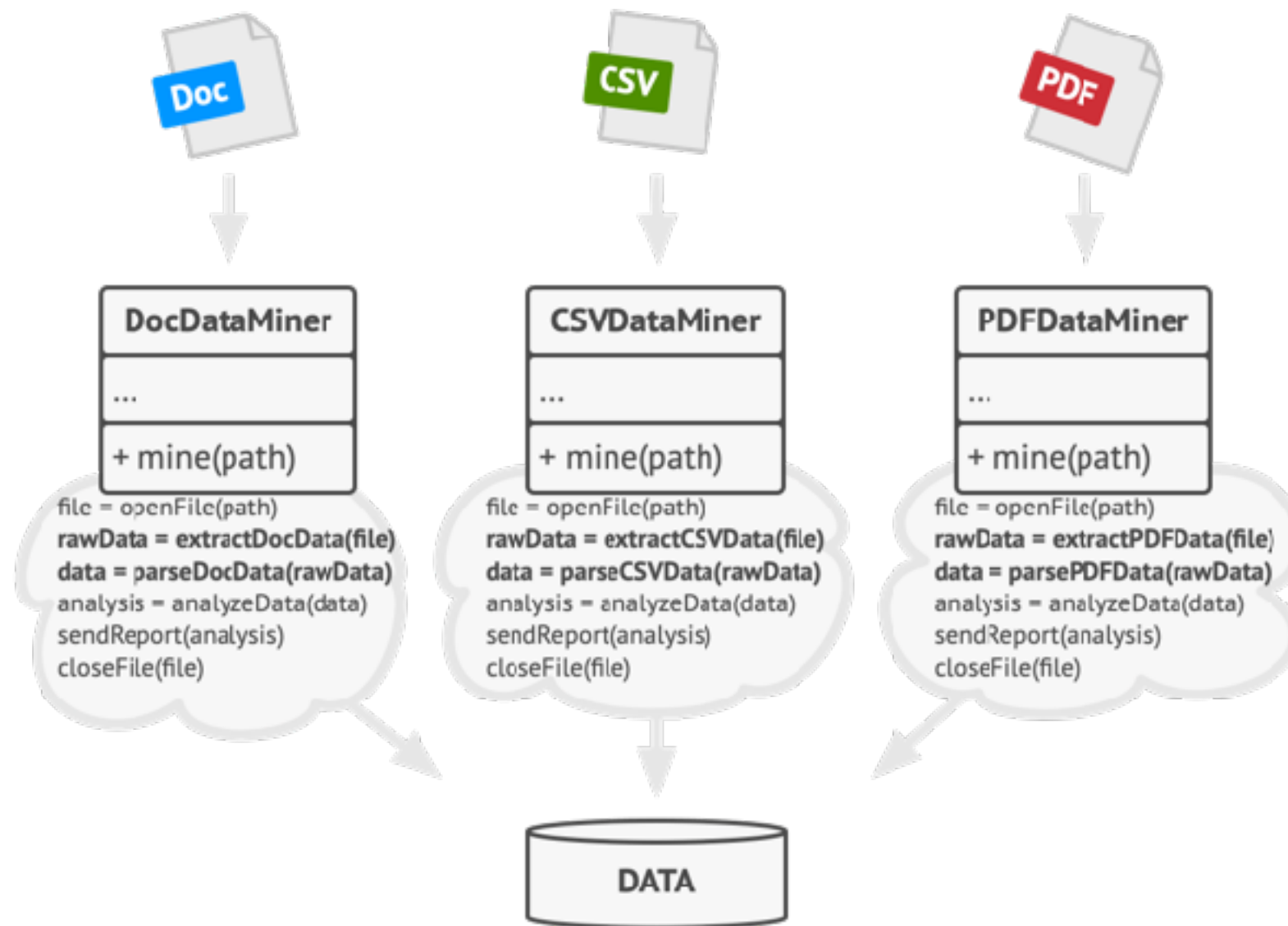
Purpose of abstract class

Create a function as base class
But can be extended by sub class

```
abstract class Report {  
    public abstract void createHeader();  
    public abstract void createBody();  
    public abstract void createFooter();  
  
    public void generate() {  
        createHeader();  
        createBody();  
        createFooter();  
    }  
}
```



Template Method Pattern



<https://refactoring.guru/design-patterns/template-method>



Interfaces

Like an extreme abstract class

All of methods in an interface are abstract by default

Not a class

No instance variables

```
interface Animal {  
    void eat();  
    void sleep();  
}
```



Using interface from class (1)

```
class Cat implements Animal {
```

```
    @Override  
    public void eat() {  
    }
```

```
    @Override  
    public void sleep() {  
    }
```

```
}
```



Using interface from class (2)

```
class Cat implements Animal {
```

```
    @Override  
    public void eat() {  
    }  
  
    @Override  
    public void sleep() {  
    }  
}
```

Need to implements all method from interface



Interface Segregation Principle

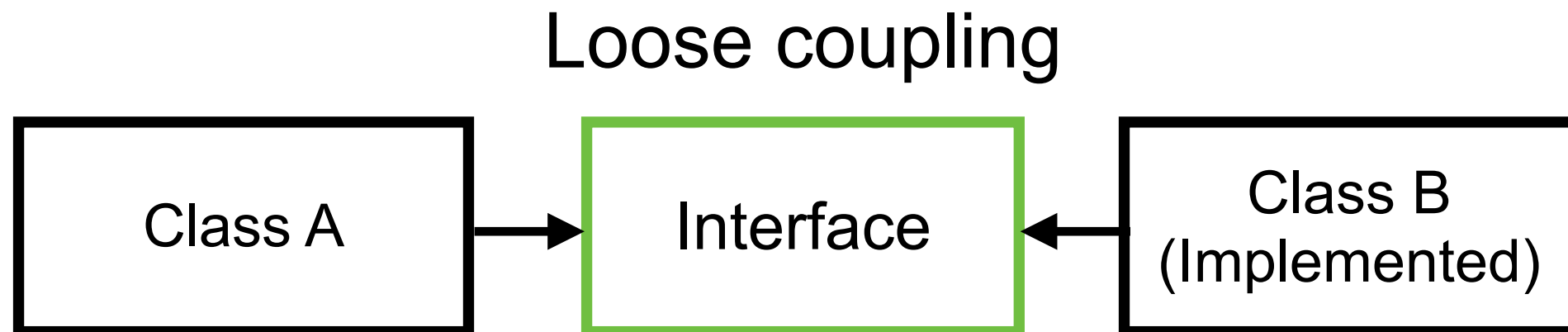
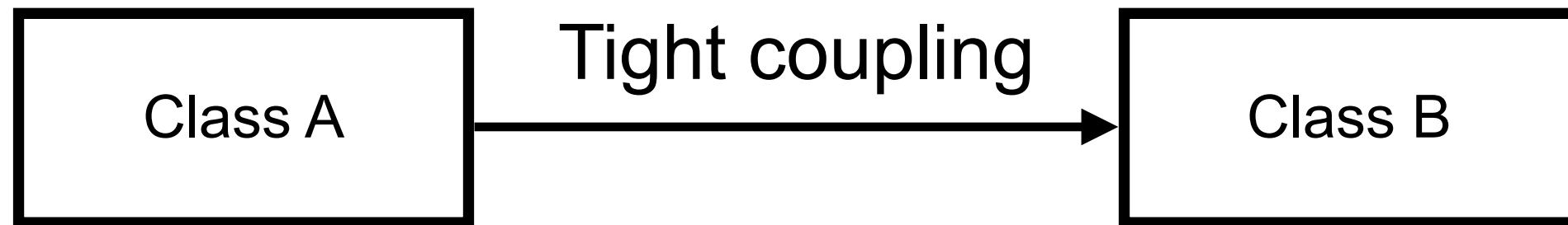


Interface Segregation Principle

You want me to plug this in *where*?



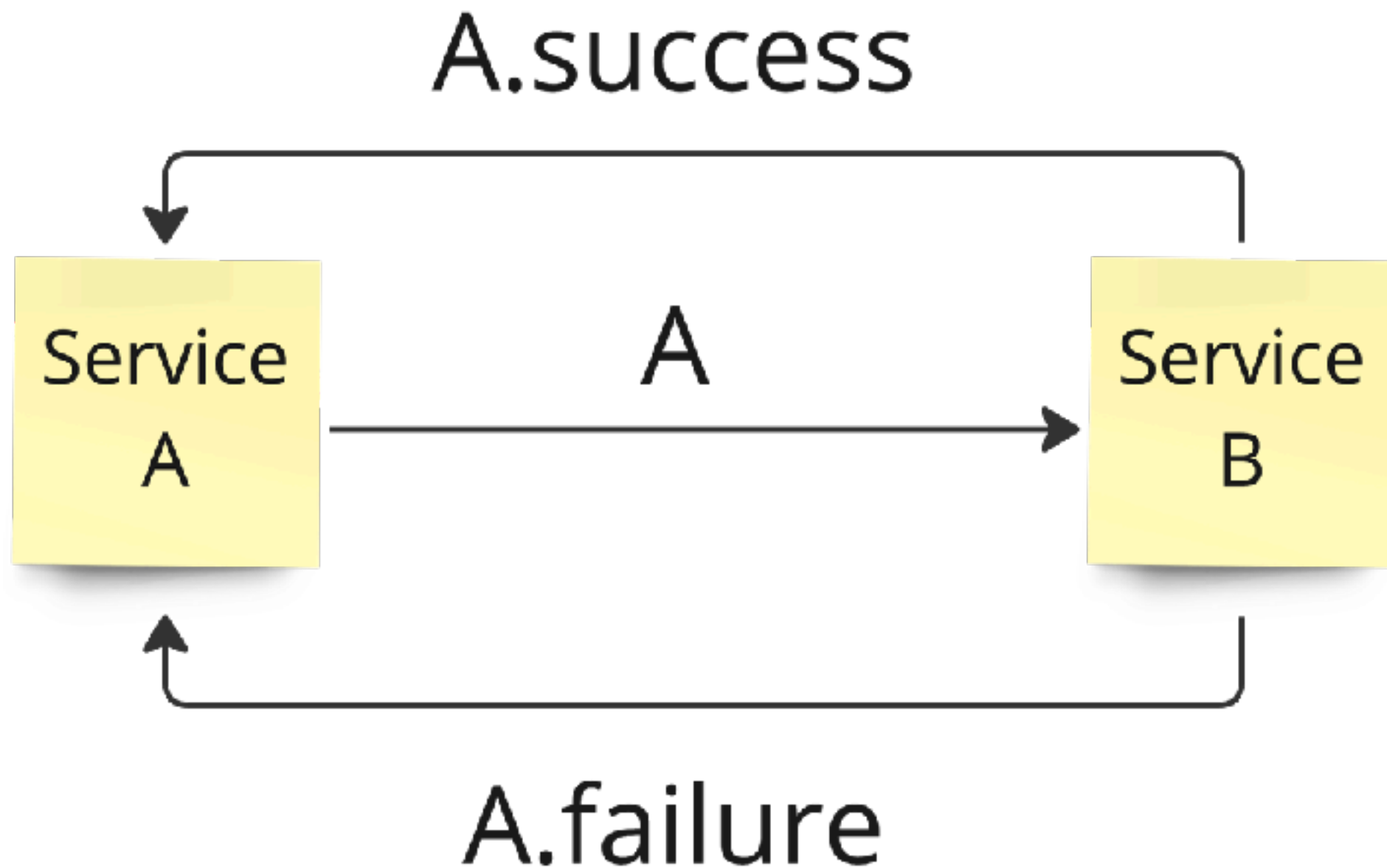
When to use interface ?



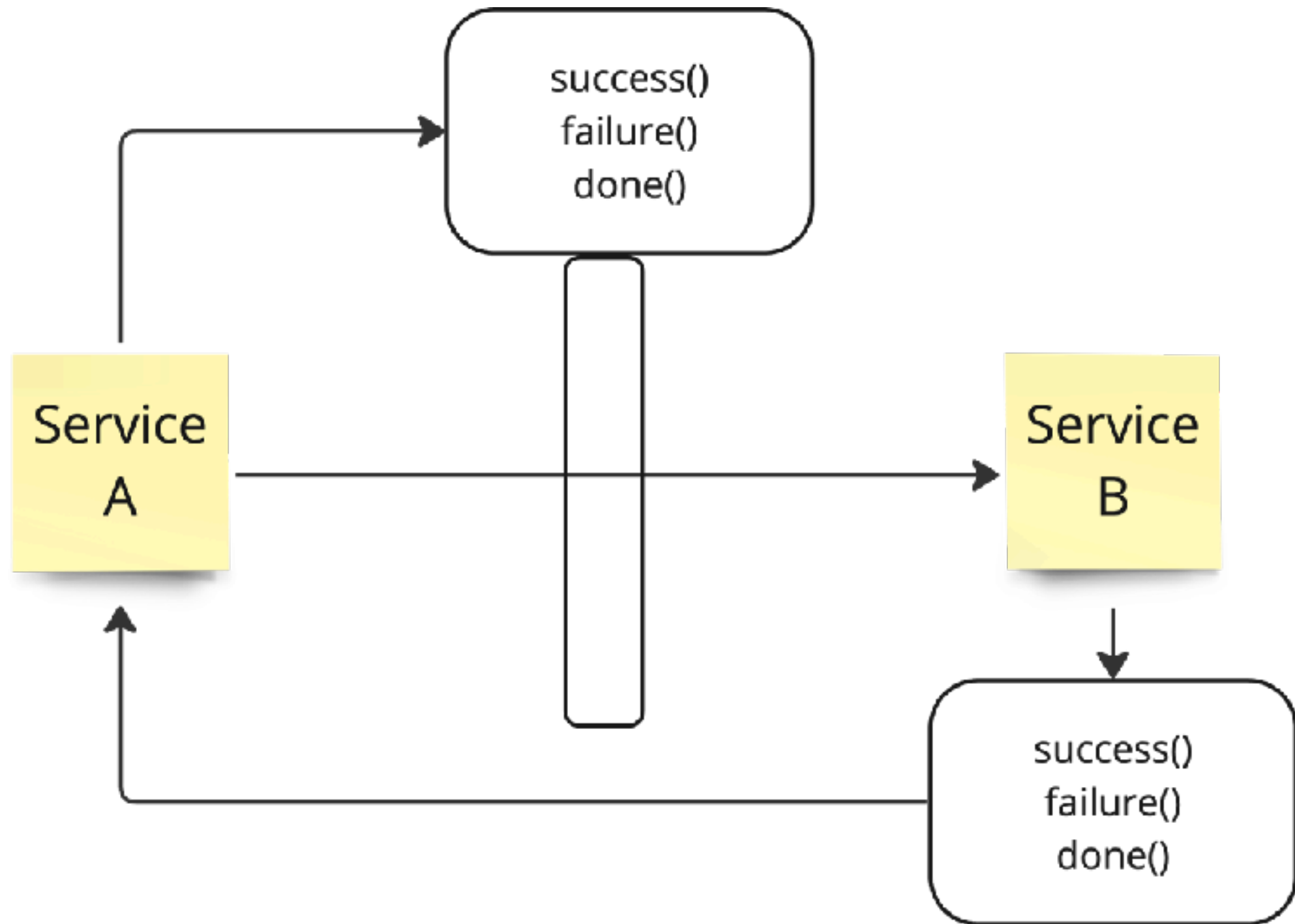
Callback with Interface



Call back with class (1)



Call back with interface (2)



Dependency Inversion Principle

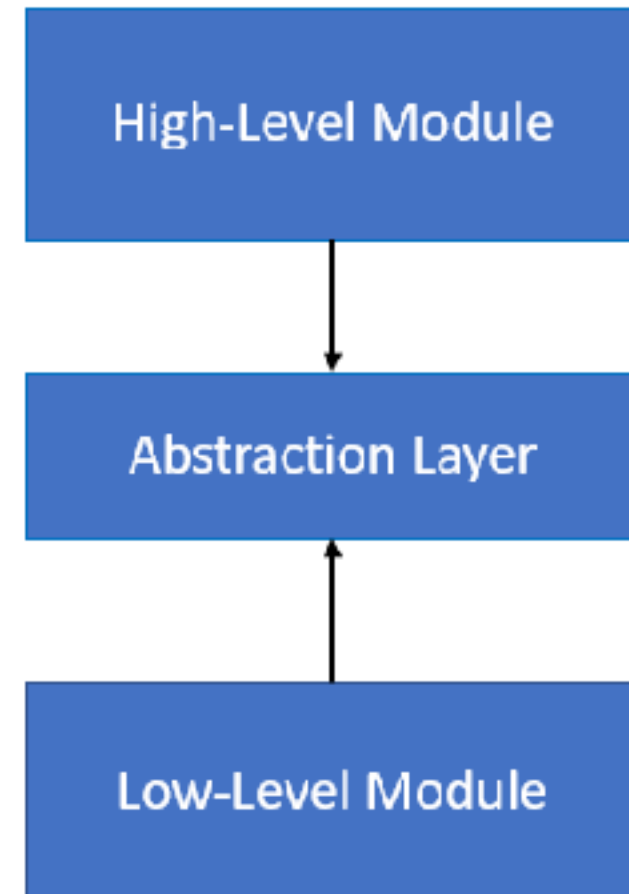
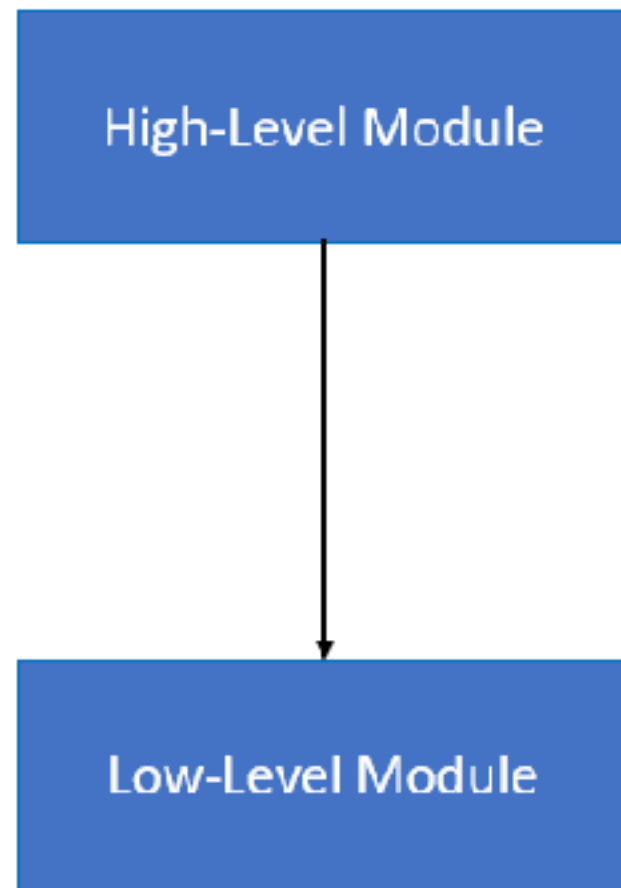


Dependency Inversion Principle

Would you solder a lamp directly to the electrical wiring in a wall?



Depend on abstraction



Java Collection Framework

<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/doc-files/coll-overview.html>



Java Collection Framework

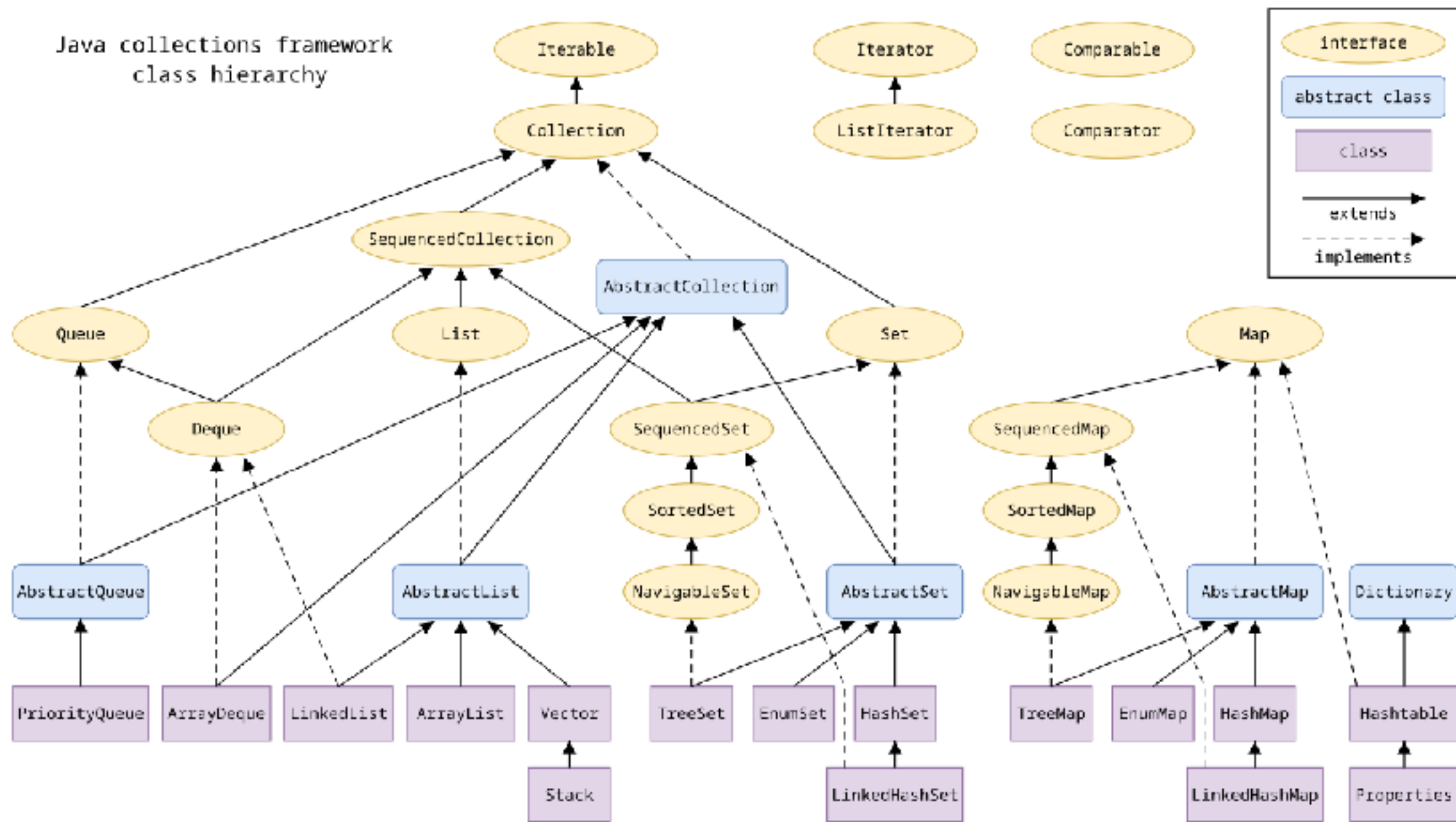
General purpose implementations

Interface	Hash Table	Resizable Array	Balanced Tree	Linked List	Hash Table + Linked List
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList		LinkedList	
Queue, Deque		ArrayDeque		LinkedList	
Map	HashMap		TreeMap		LinkedHashMap

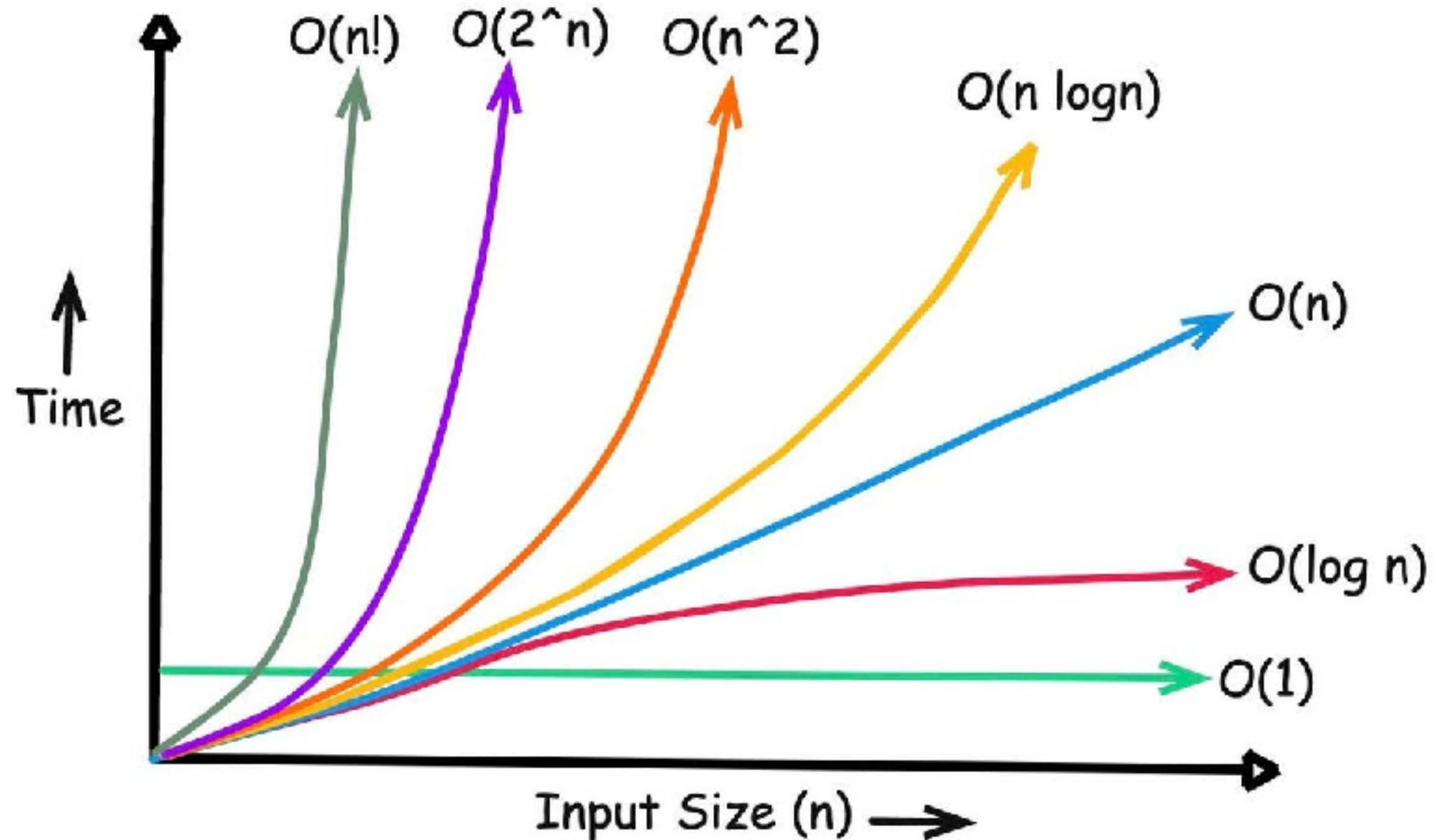
<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/doc-files/coll-overview.html>



Java Collection Framework



Big O Notation

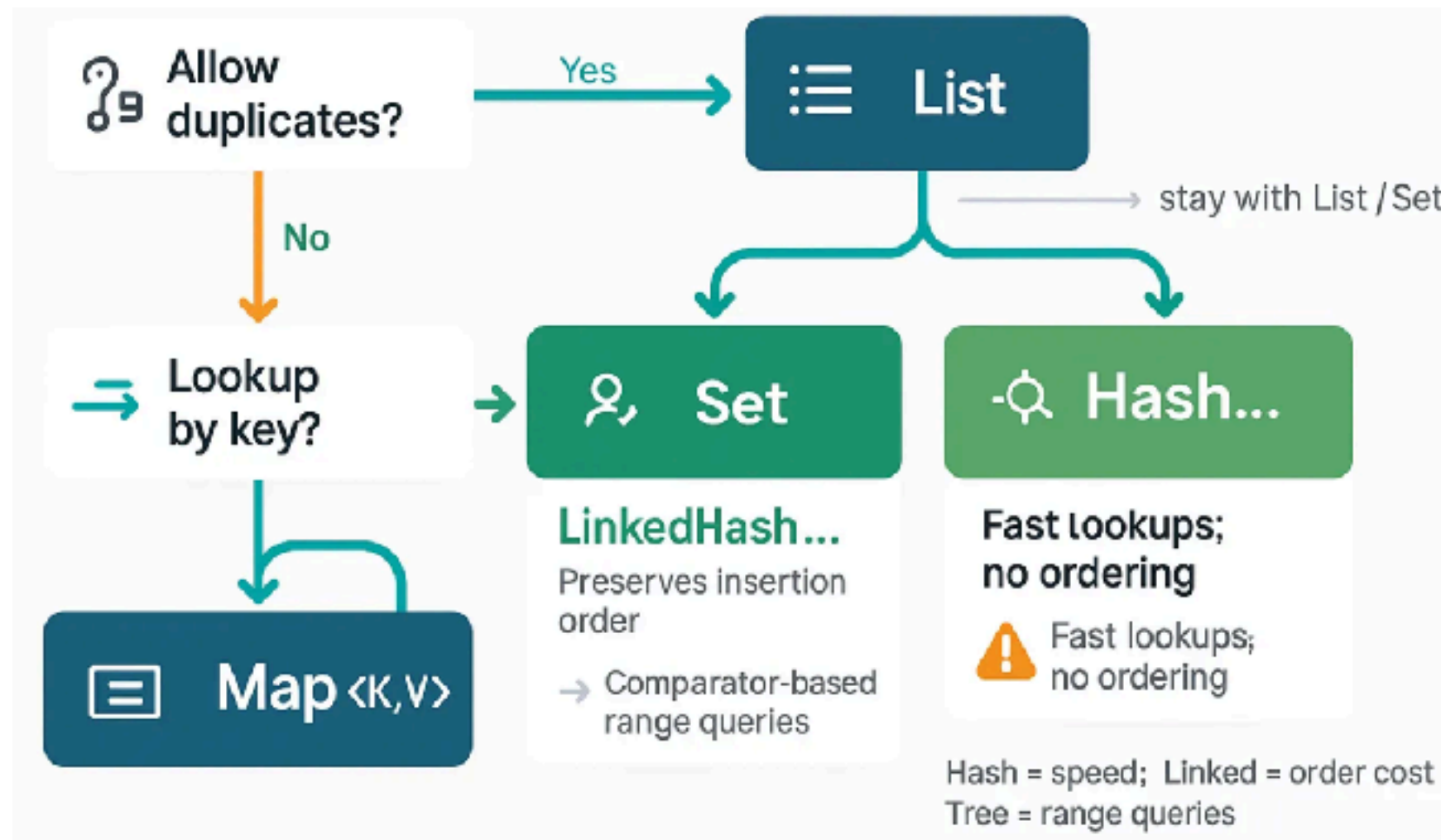


Working with use cases



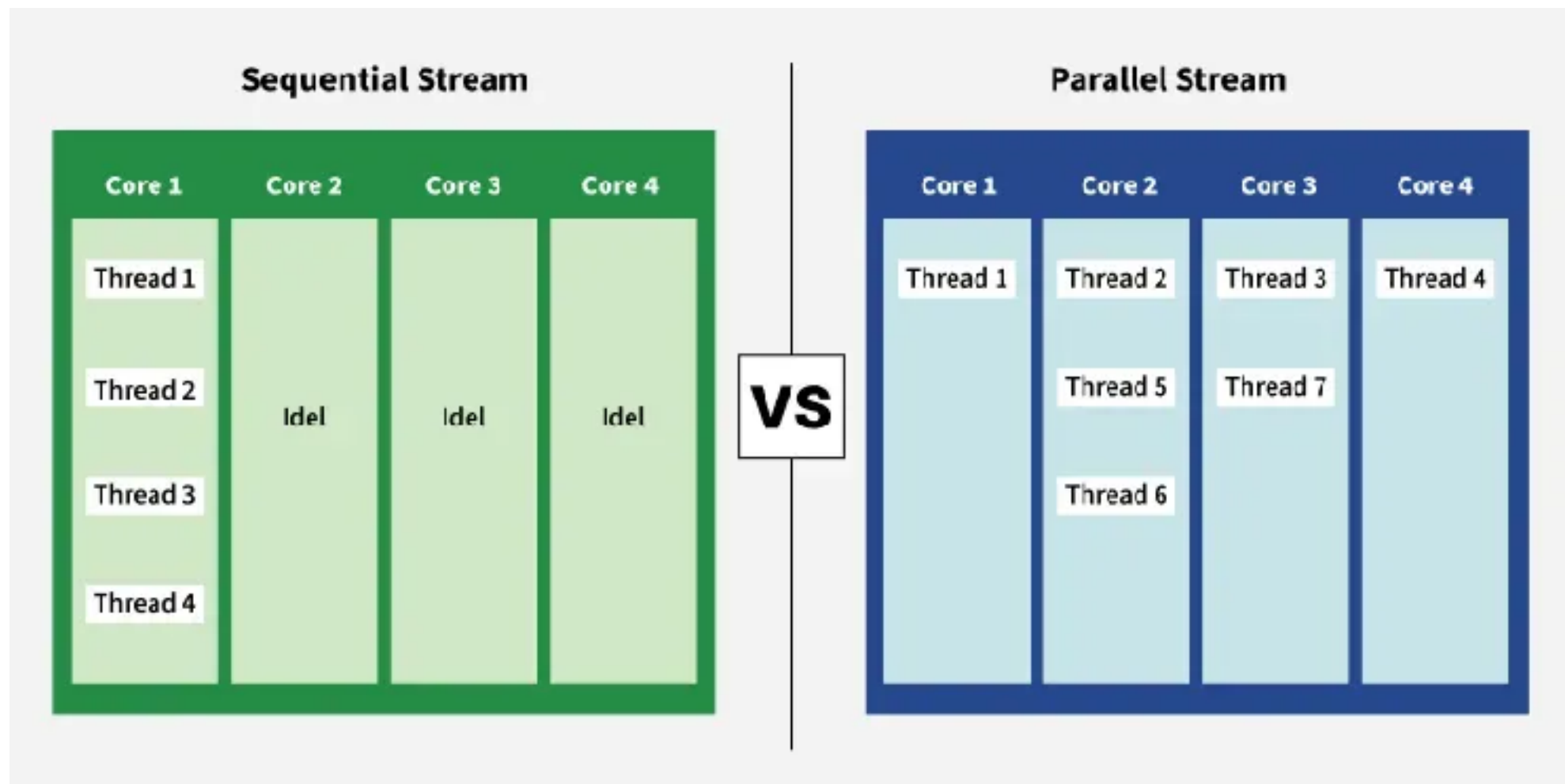
List vs Map

Lookup optimization



Java Stream

Sequential vs Parallel



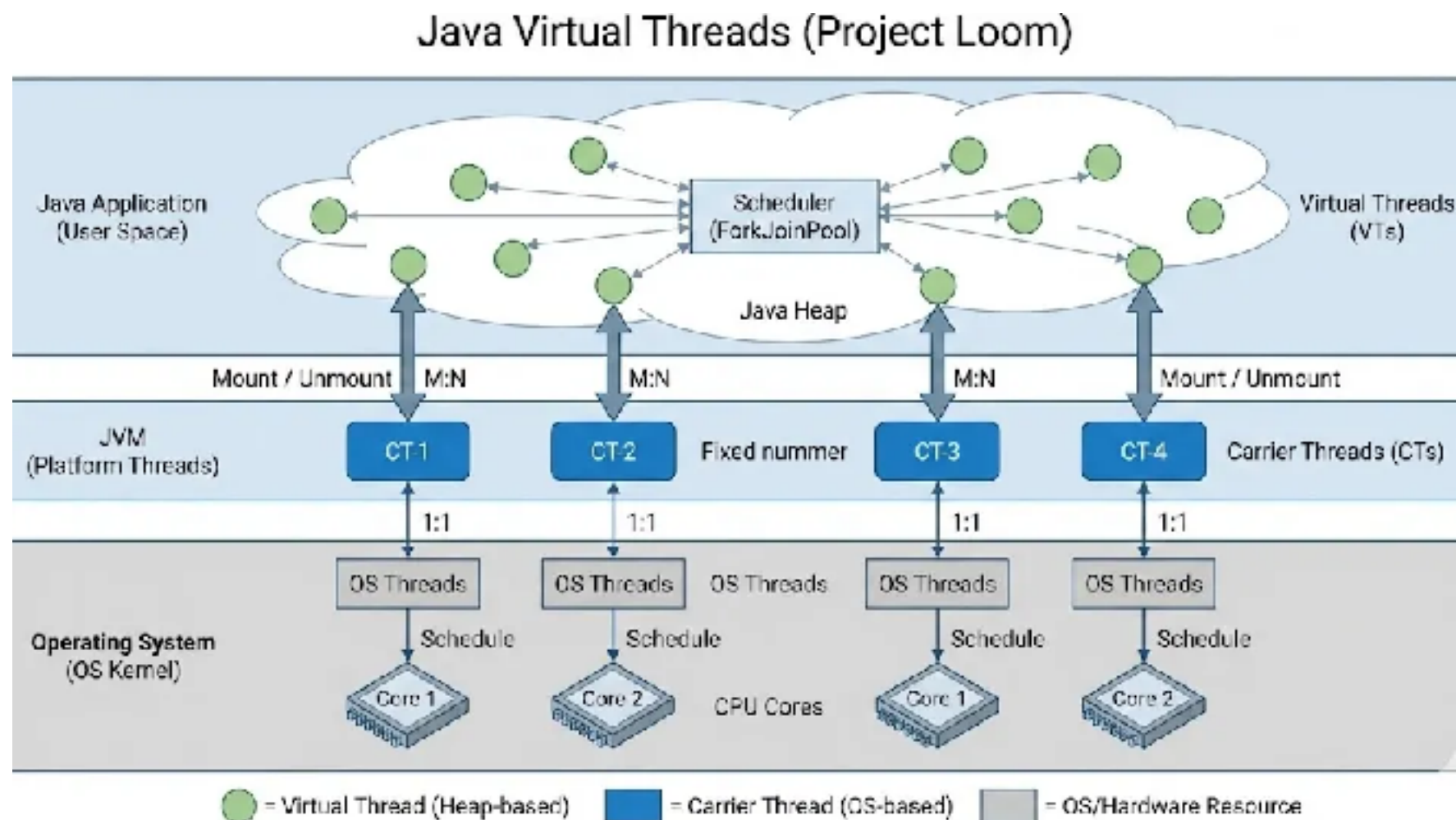
<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/stream/Stream.html>



Java Virtual Threads (Project Loom)

Java 21

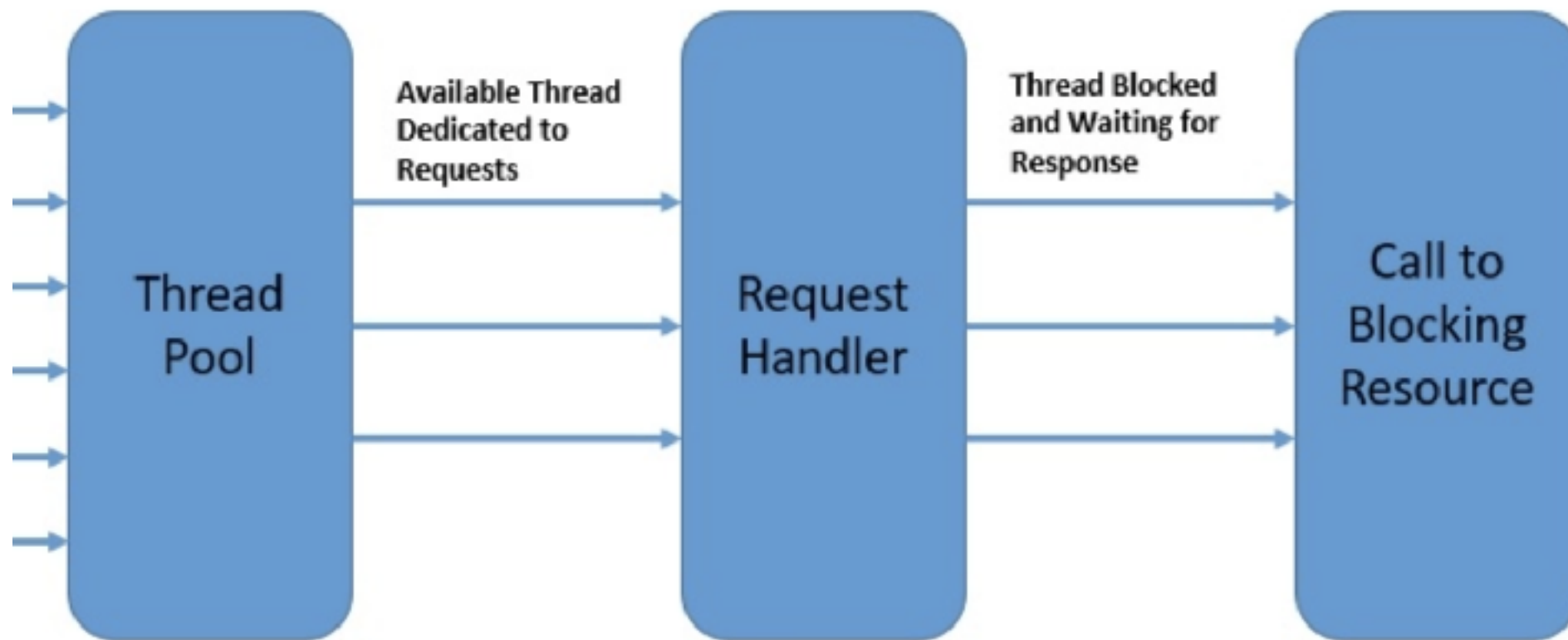
Lightweight, JVM-managed thread



<https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html>



Spring Boot with Virtual Thread !!



<https://www.somkiat.cc/virtual-thread-in-spring-boot-3/>

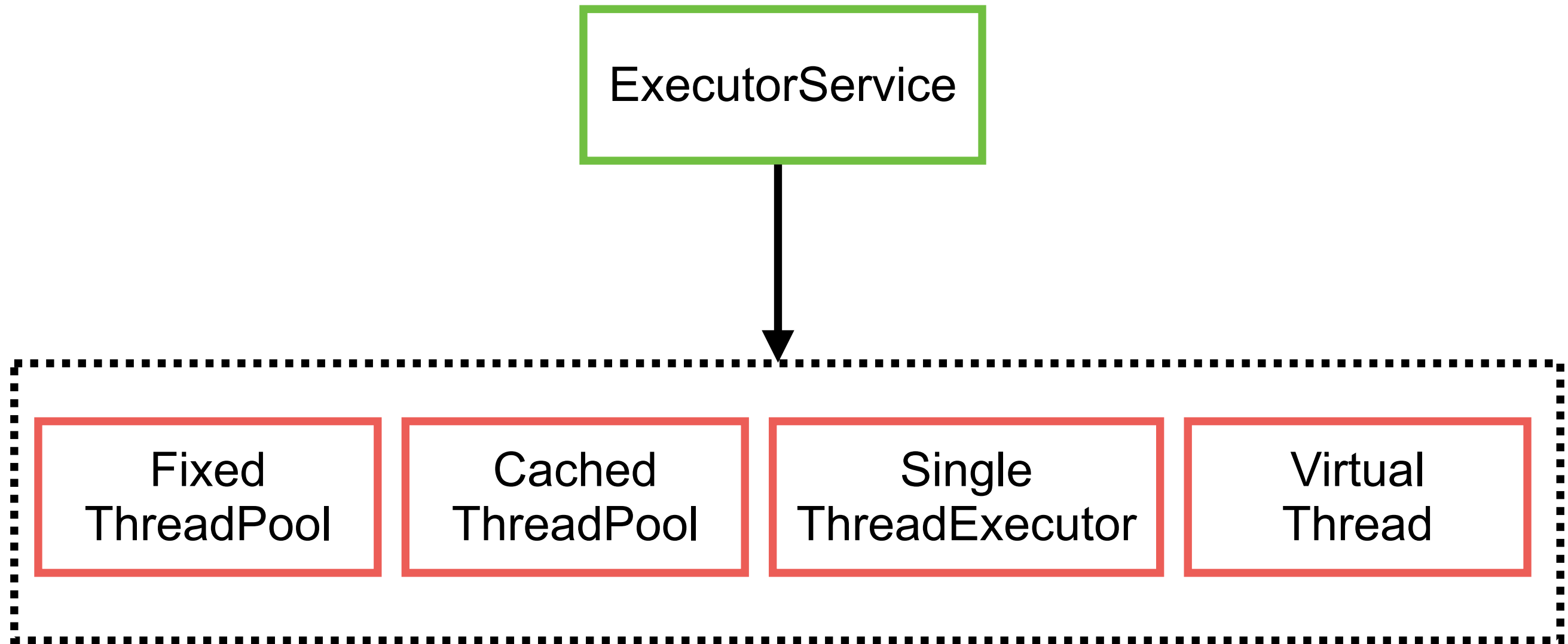


Platform vs Virtual Thread

Feature	Platform thread	Virtual thread (Java21+)
Mapping	1:1 with OS threads	M:N Many virtual threads to few OS threads
Creation cost	High memory and CPU	Low
Memory footprint	1 MB per thread (Stack size)	< 1 KB (Stored on heap)
Use case	CPU-intensive tasks	I/O intensive tasks (blocking call)
Scalability	Limit by OS resources	Scalable to millions
Pooling ?	Required to prevent resource exhaustion	Create a new one per task



High level concurrency framework





Exception Handling

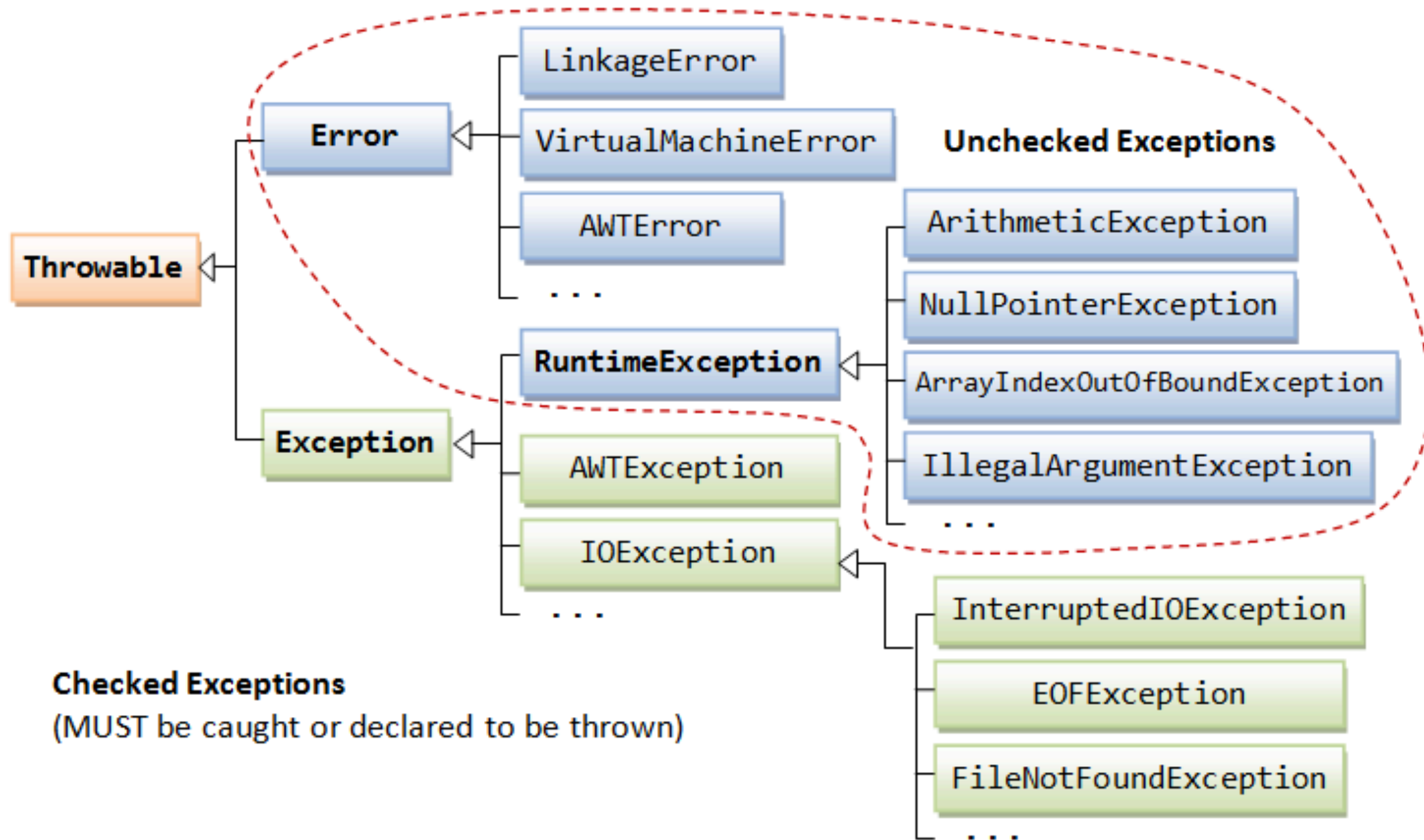


Types of Exception

- Compile-time
- Runtime
- Logical



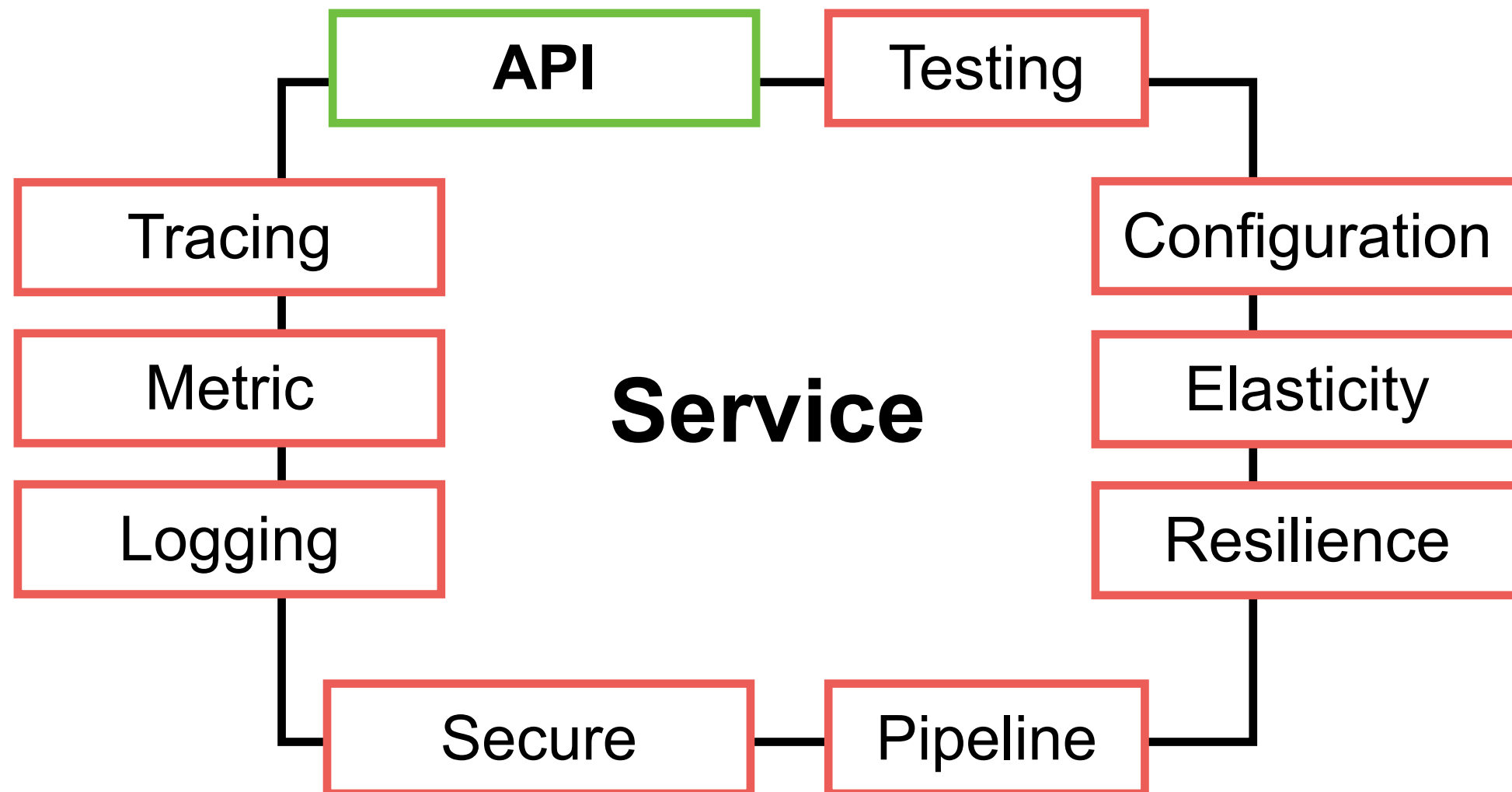
Exception Classes



REST API with Spring Boot



Properties of Service



APIs

Application Programming Interface

REST API

gRPC

Messaging



REST

REpresentation State Transfer

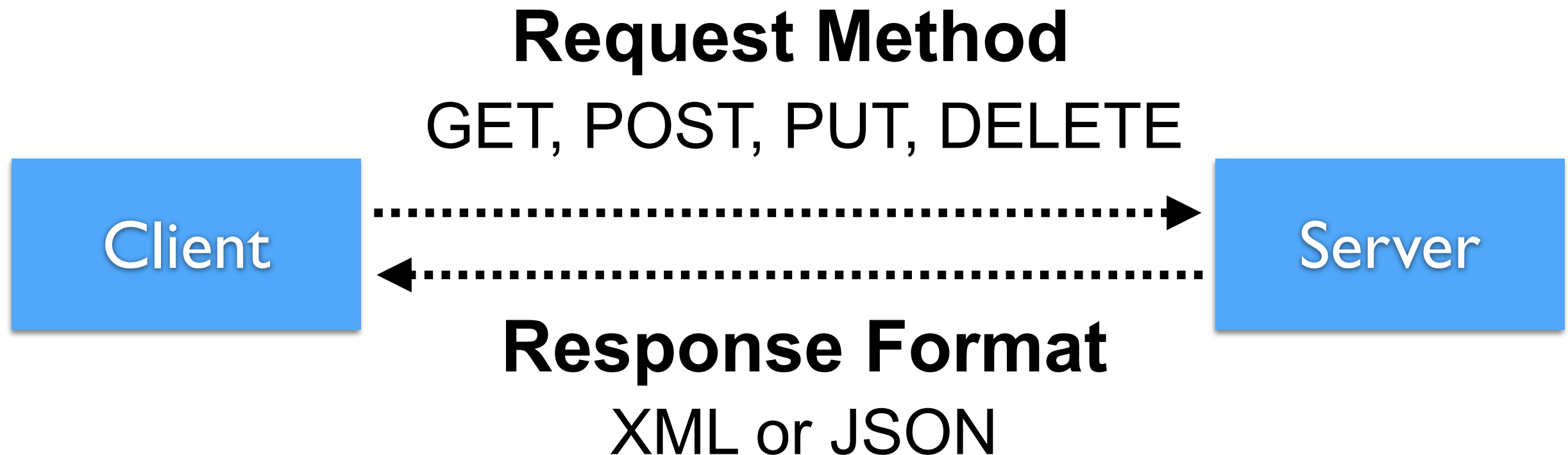
The style of **software architecture** behind
RESTful services

Defined in 2000 by Roy Fielding

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm



REST Request & Response



HTTP Methods meaning

Method	Meaning
GET	Read data
POST	Create/Insert new data
PUT/PATCH	Update data or insert if a new id
DELETE	Delete data



Response format ?



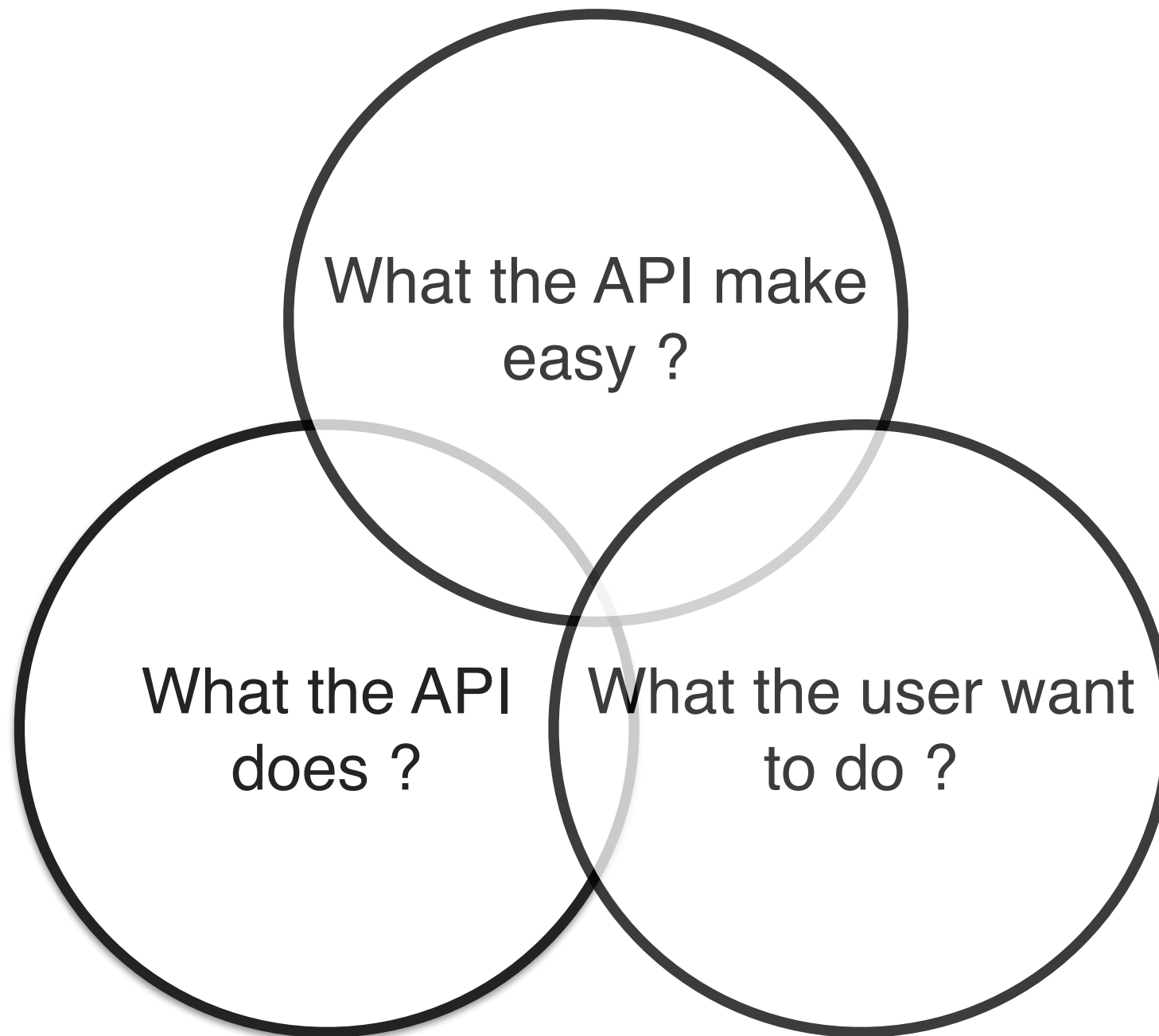
Status Code

Code	Description
1xx	Information
2xx	Successful
3xx	Redirection
4xx	Client error
5xx	Server error

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>



Good APIs ?



Principles of API Design

Consistency

Statelessness

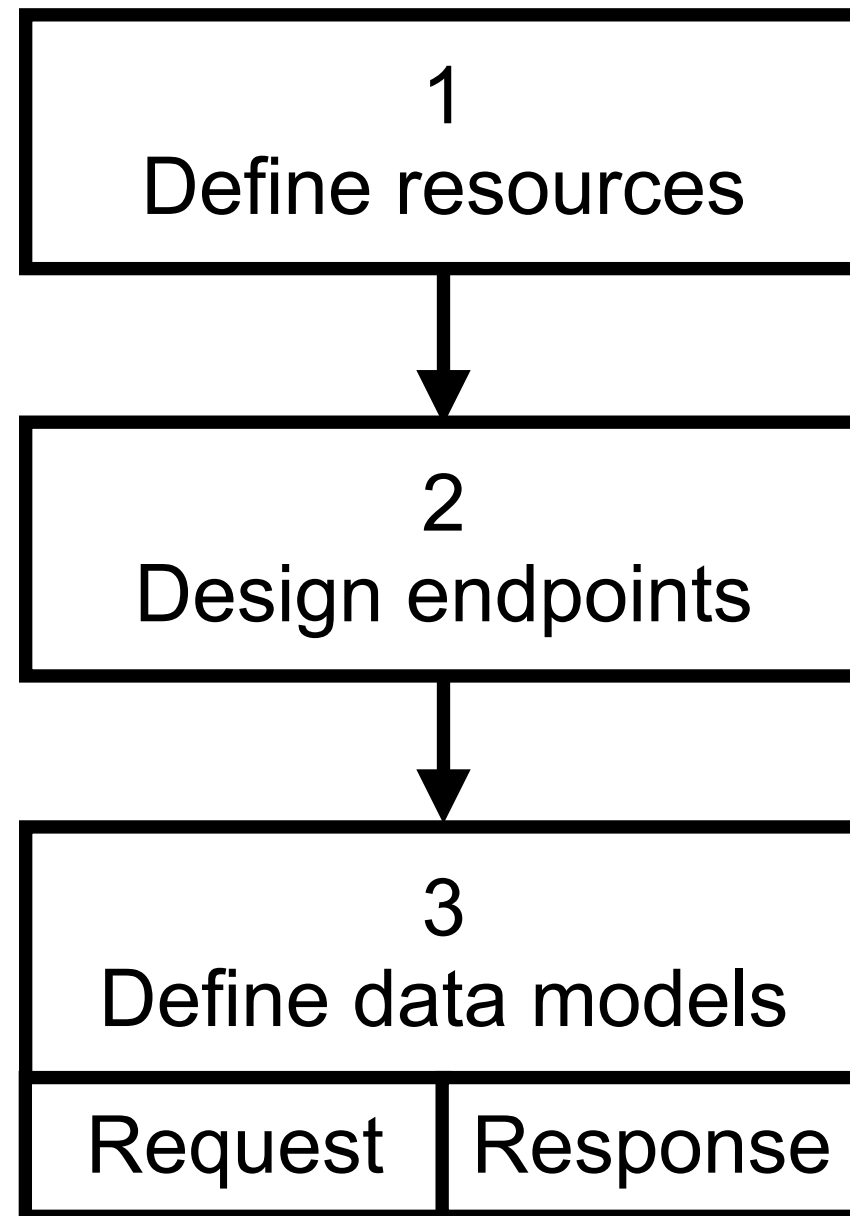
Resource-oriented design

Use standard HTTP methods

Versioning



Steps to Design APIs



More !!

Authentication
Authorization

Pagination and
filtering

Rate Limiting

Pagination and
filtering

Error handling

Documentation

Testing

Monitoring and
Observability



Implementation

Coding

Testing

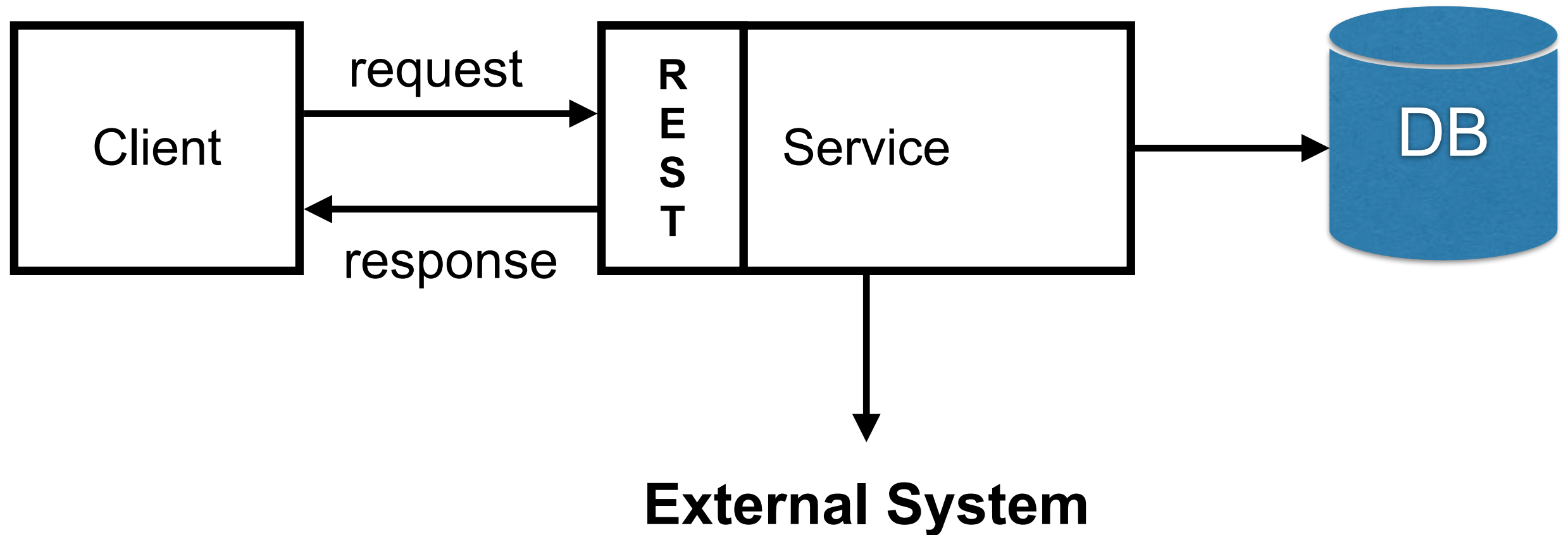
Observability

Performance

Security



Structure of Service



Spring Boot Structure (1)

Separate by function/domain/feature

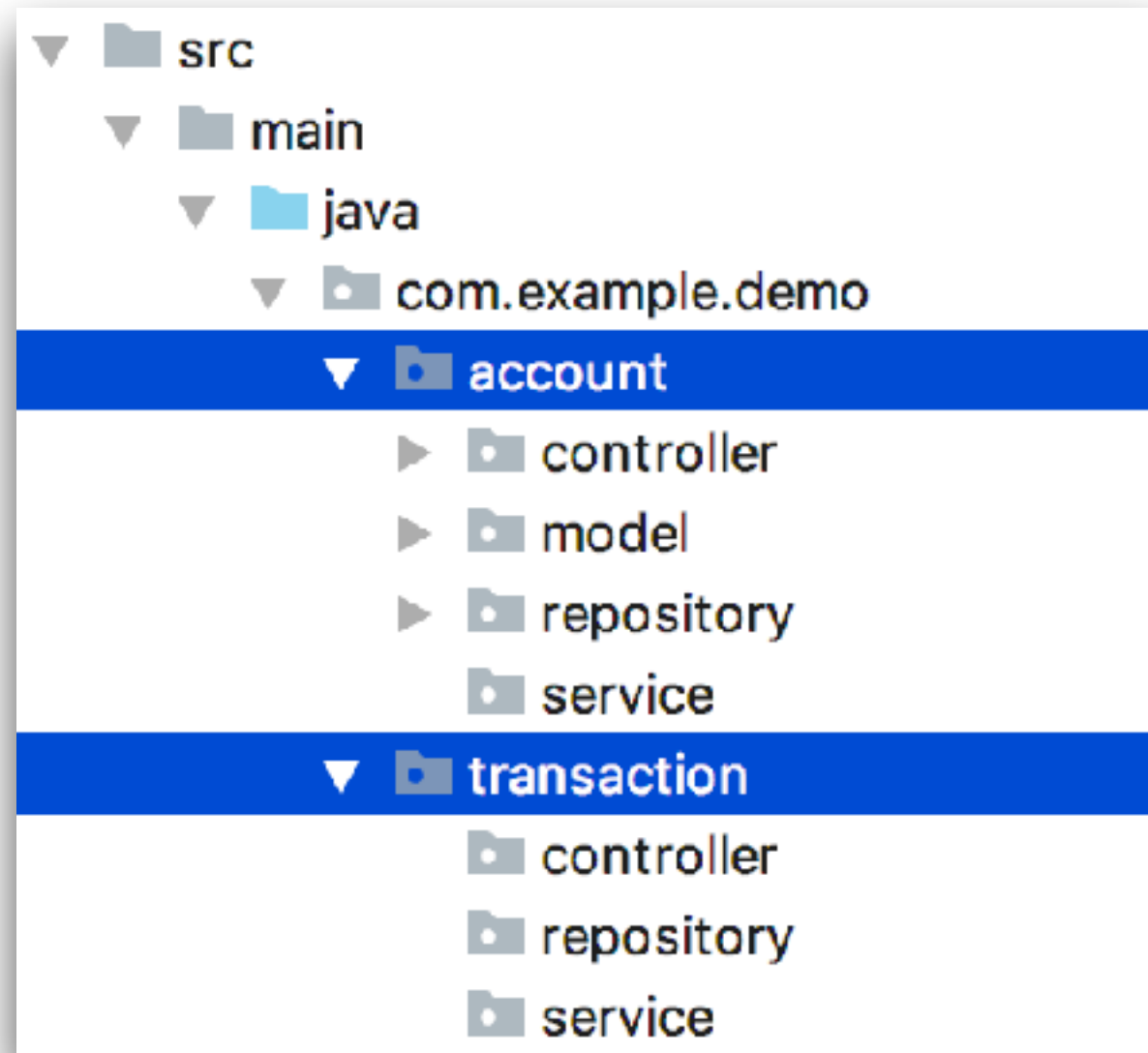
```
feature1
|   - controller
|   - service
|   - repository
feature2
|   - controller
|   - service
|   - repositor
```

<https://docs.spring.io/spring-boot/reference/using/structuring-your-code.html>

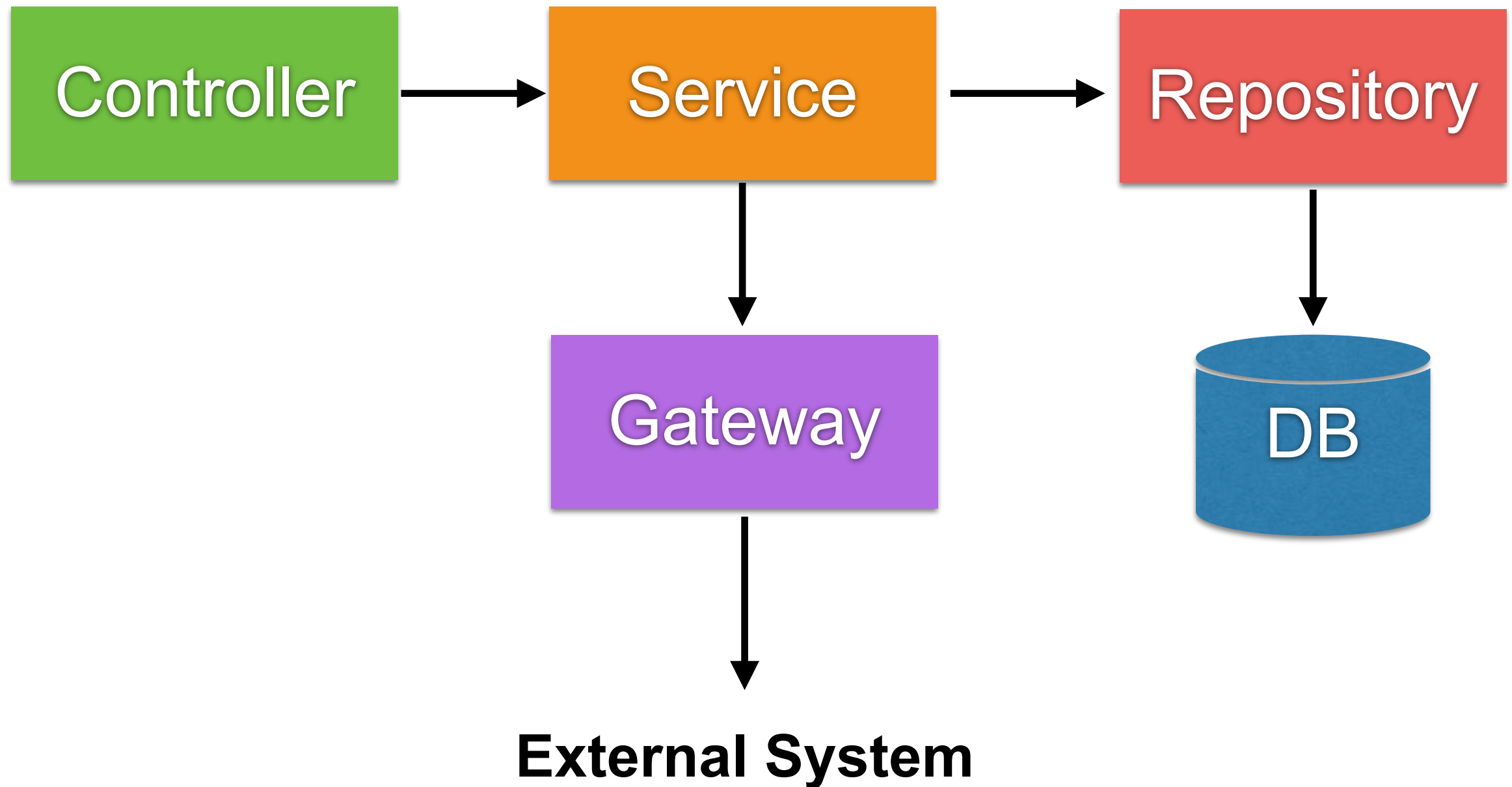


Spring Boot Structure (2)

Separate by function/domain/feature



Structure of Spring Boot project



Controller

Request and Response

Validation input

Delegate to others classes such as service and repository



Service

Control the flow of main business logics

Manage database transaction

Don't reuse service



Repository

Manage data in data store such as RDBMS and NoSQL



Gateway

Call external service via network such as
WebServices and RESTful APIs



Convert JSON to POJO

IntelliJ IDEA

Code Tools

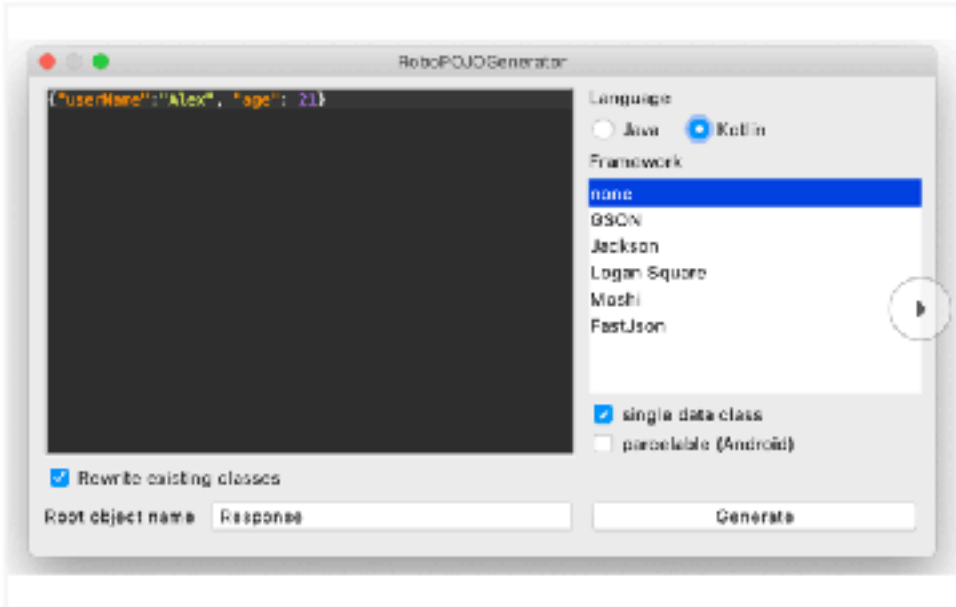
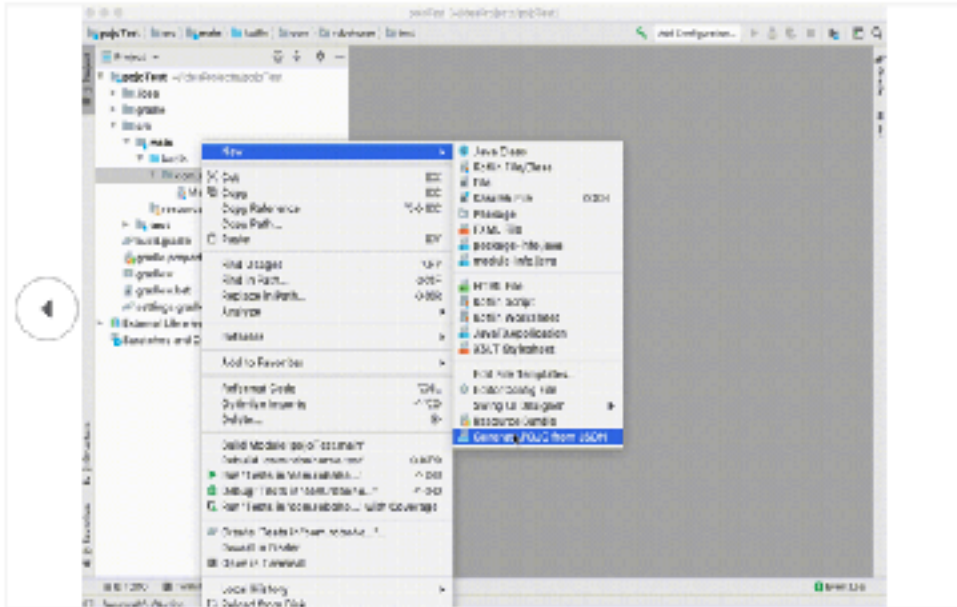
RoboPOJOGenerator

★★★★★
Vadim Shcheney

[Overview](#) [Versions](#) [Reviews](#)

Install to IntelliJ IDEA 2024.14

Compatible with IntelliJ IDEA (Ultimate, Community), Android Studio



IntelliJ Idea and Android Studio plugin for JSON to POJO transformation.

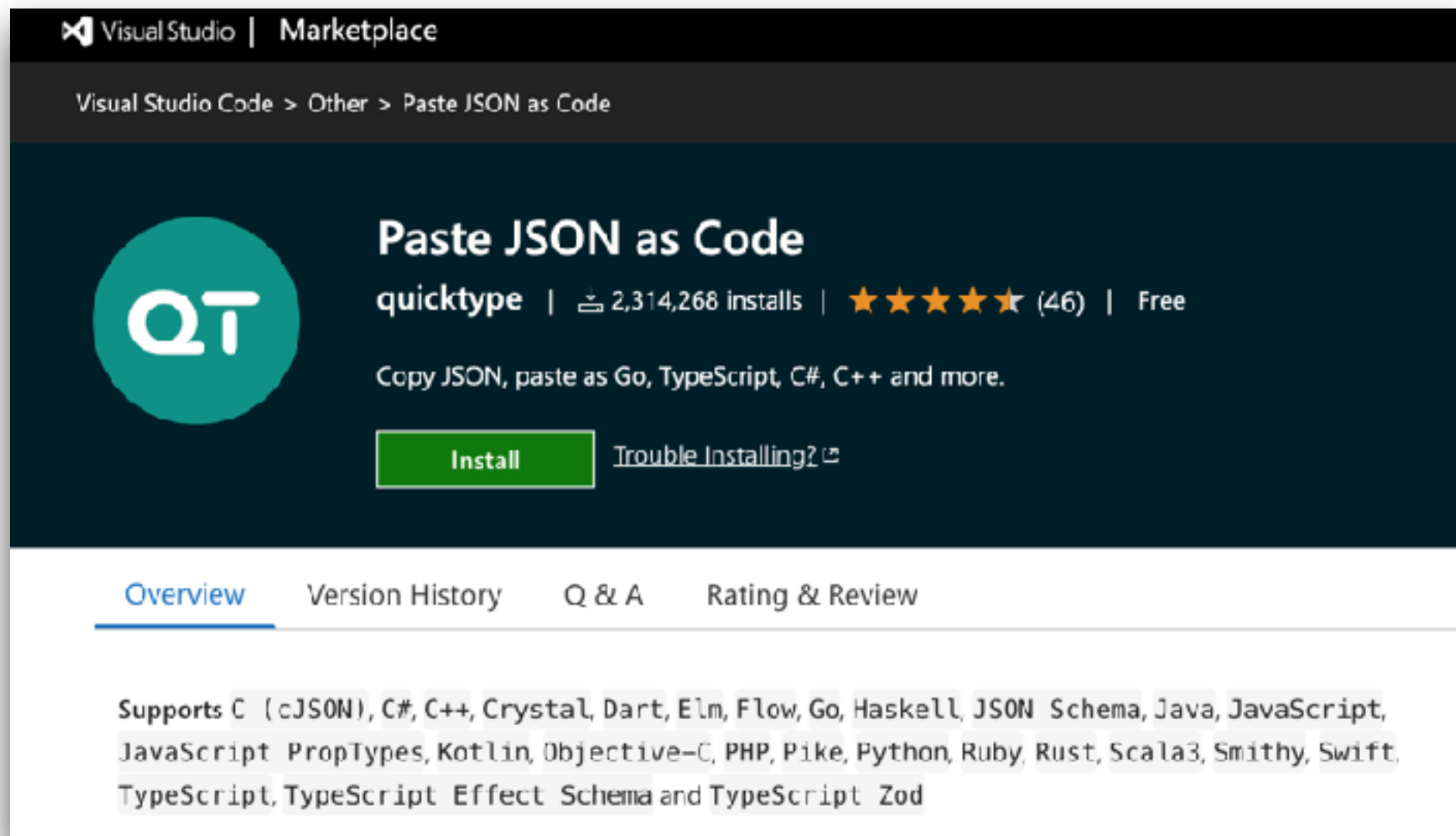
Generates Java, Java Records and Kotlin POJO files from JSON: GSON, FastJSON, AutoValue (GSON), Logan Square, Jackson, Lombok, Jakarta JSON Binding, empty annotations template. Supports: primitive types,

<https://plugins.jetbrains.com/plugin/8634-robopojogenerator>



Convert JSON to POJO

VS Code



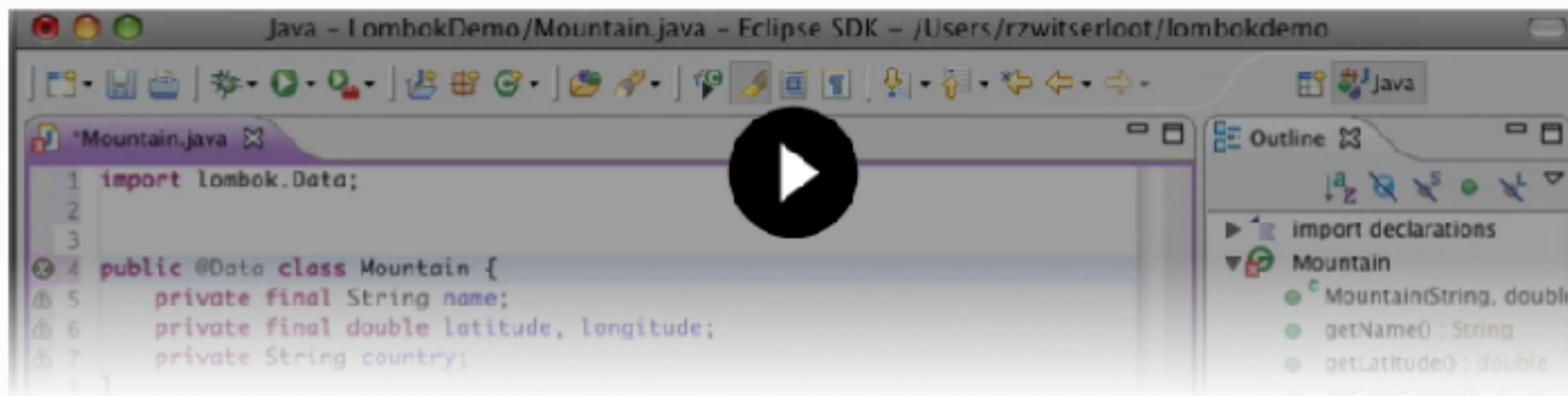
<https://marketplace.visualstudio.com/items?itemName=quicktype.quicktype>



Using Lombok

Project Lombok

Project Lombok is a Java library that automatically plugs into your editor and build tools, splicing up your Java. Never write another getter or equals method again, with one annotation your class has a fully featured builder, Automate your logging variables, and much more.



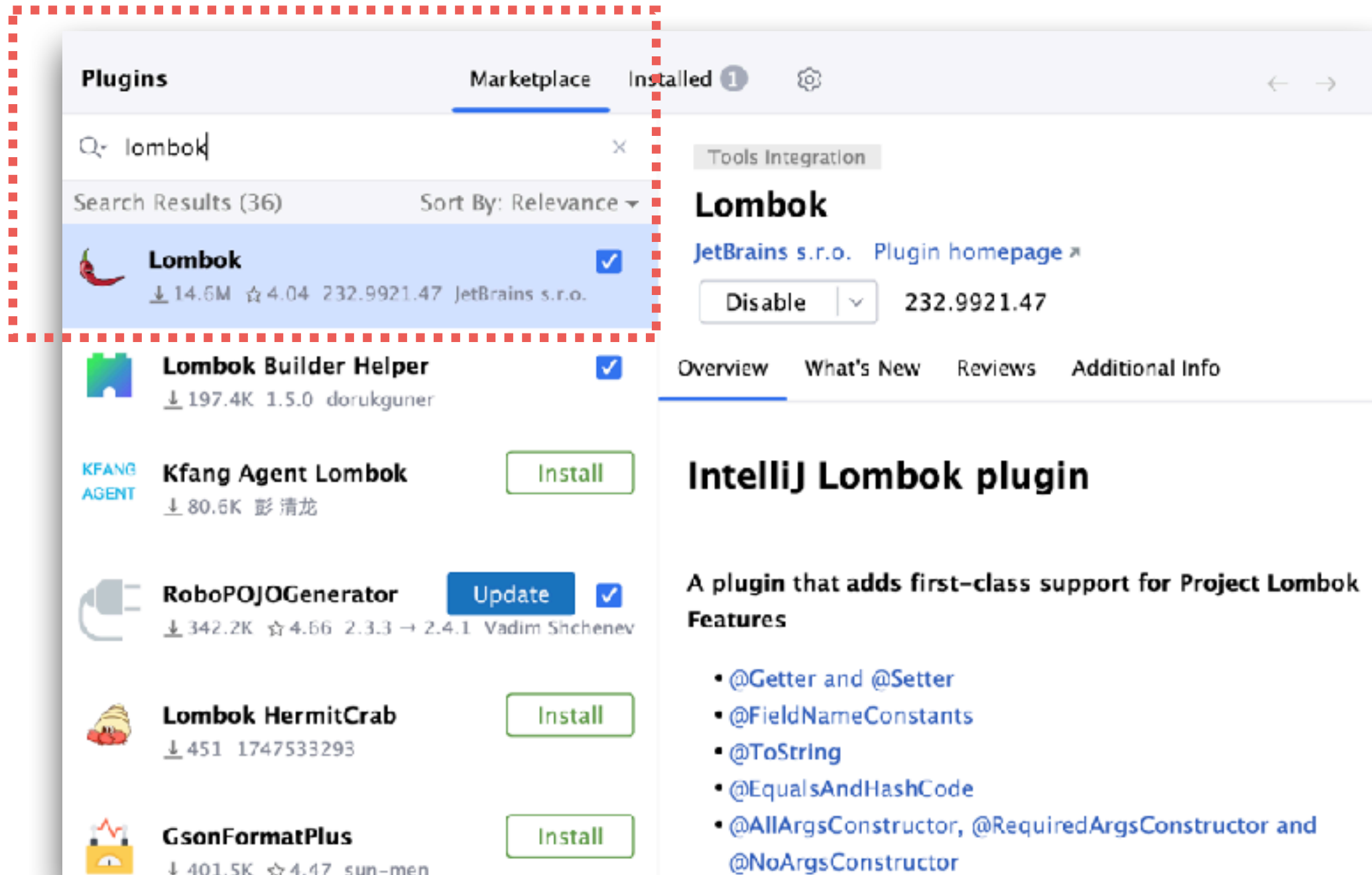
Show me a text and images based explanation and tutorial instead!

Project Lombok is **powered by:**

<https://projectlombok.org/>



IntelliJ IDEA plug-in



<https://plugins.jetbrains.com/plugin/6317-lombok>



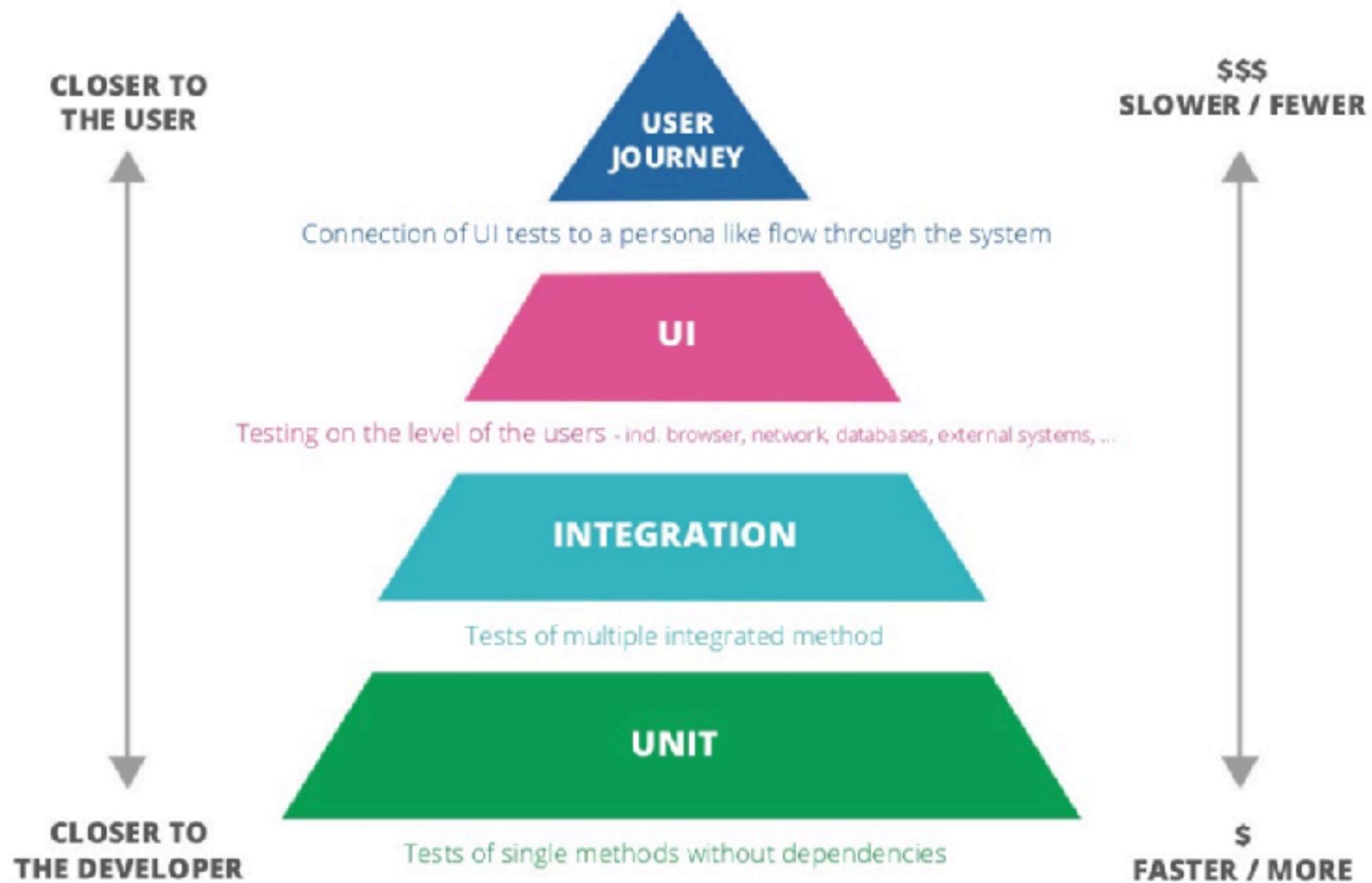
Use Lombok

```
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class UserRequest {
    private String email;
    private String password;
}
```



Testing with Spring Boot





Spring Boot Testing

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-webmvc-test</artifactId>  
  <scope>test</scope>  
</dependency>
```



6



<https://docs.spring.io/spring-boot/docs/current/reference/html/features.html#features.testing>



Testing in Spring Boot ?

@SpringBootTest

@WebMvcTest

@JsonTest

@DataJpaTest

@RestClientTest

<https://docs.spring.io/spring-boot/reference/testing/spring-boot-applications.html>



Testing in Spring Boot

@SpringBootTest

@WebMvcTest

@JsonTest

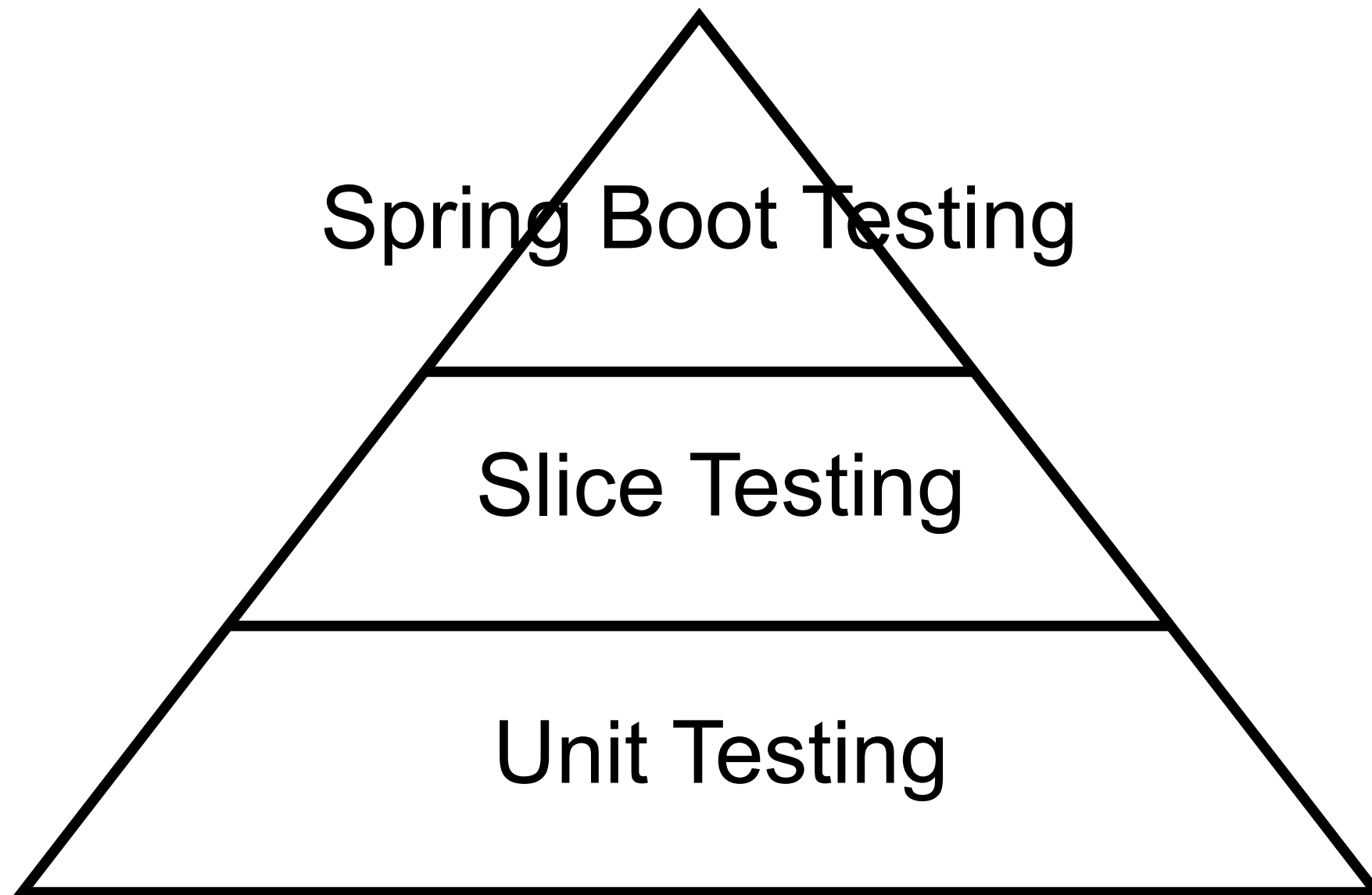
@DataJpaTest

@RestClientTest

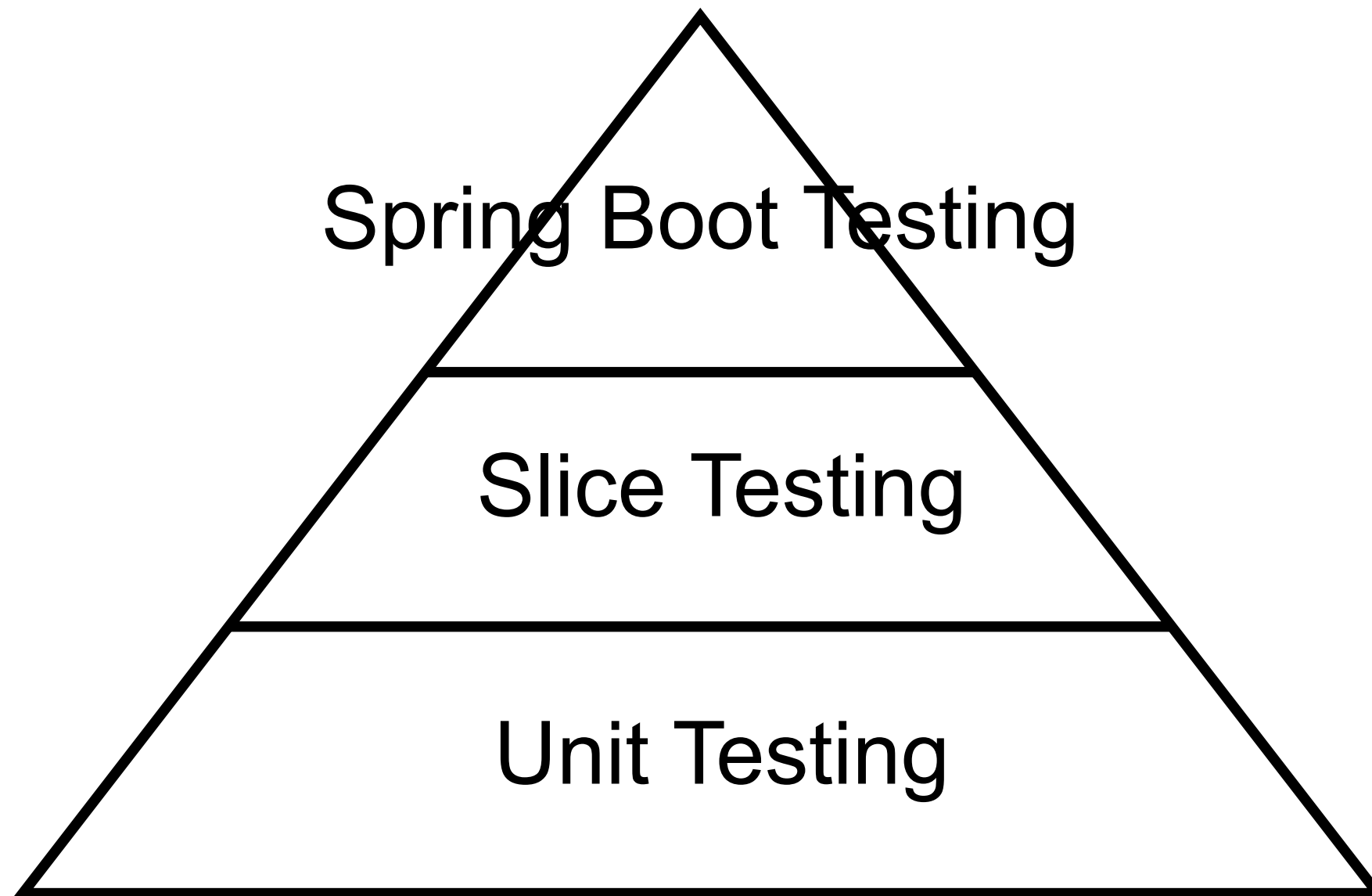
Slice testing



Testing in Spring Boot



Testing in Spring Boot



Controller testing

1. Spring Boot Testing
2. Slice Testing with MockMvc
3. Unit Testing



1. SpringBootTest

Application context

Controller Advice

Controllers

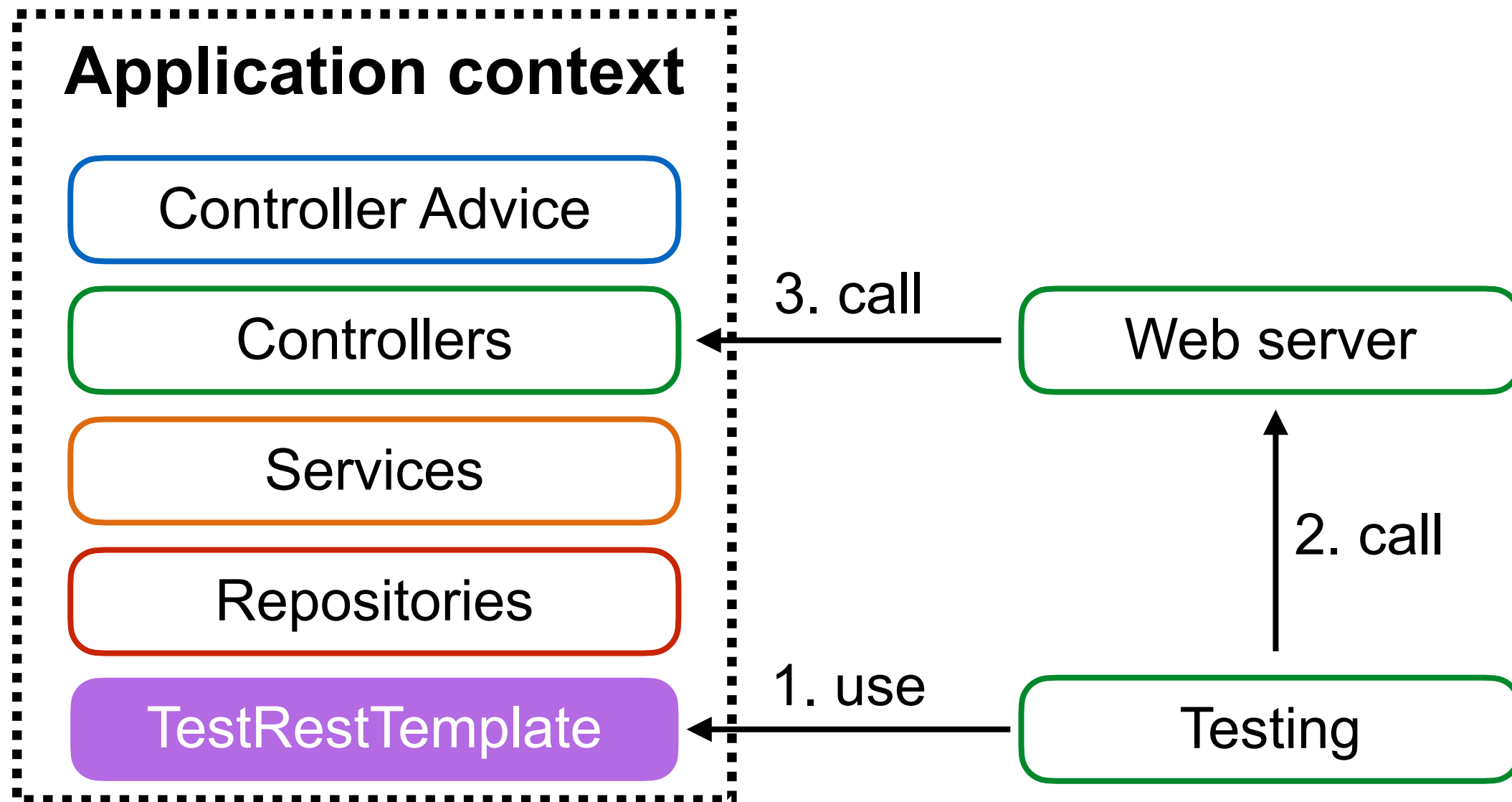
Services

Repositories

<https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/#features.testing>



1. SpringBootTest



SpringBootTest #1

```
@RunWith(SpringRunner.class)
@SpringBootTest(webEnvironment
    = SpringBootTest.WebEnvironment.RANDOM_PORT)
public class HelloControllerTest {
```

```
    @Autowired
    private TestRestTemplate testRestTemplate;
```

```
    @Test
    public void sayHi() {
        // Action :: Call controller
        Hello actualResult
            = testRestTemplate.getForObject("/hello/somkiat",
                                           Hello.class);

        // Assertion :: Check result with expected result
        assertEquals("Hello, somkiat", actualResult.getMessage());
    }
}
```



SpringBootTest #2

```
@RunWith(SpringRunner.class)
@SpringBootTest(webEnvironment
    = SpringBootTest.WebEnvironment.RANDOM_PORT)
public class HelloControllerTest {

    @Autowired
    private TestRestTemplate testRestTemplate;

    @Test
    public void sayHi() {
        // Action :: Call controller
        Hello actualResult
            = testRestTemplate.getForObject("/hello/somkiat",
                                           Hello.class);

        // Assertion :: Check result with expected result
        assertEquals("Hello, somkiat", actualResult.getMessage());
    }
}
```



SpringBootTest #3

```
@RunWith(SpringRunner.class)
@SpringBootTest(webEnvironment
    = SpringBootTest.WebEnvironment.RANDOM_PORT)
public class HelloControllerTest {
```

```
    @Autowired
    private TestRestTemplate testRestTemplate;
```

```
    @Test
    public void sayHi() {
        // Action :: Call controller
        Hello actualResult
            = testRestTemplate.getForObject("/hello/somkiat",
                                           Hello.class);

        // Assertion :: Check result with expected result
        assertEquals("Hello, somkiat", actualResult.getMessage());
    }
```

```
}
```



Error handling



Working with Error Responses

ErrorResponse

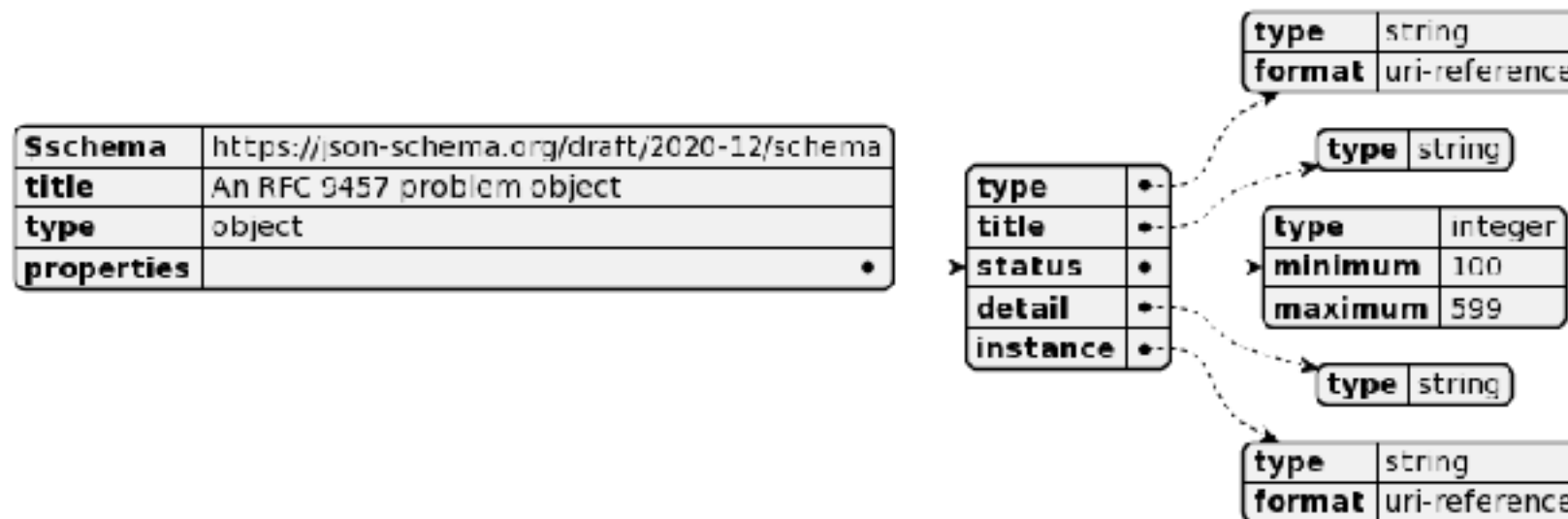
ProblemDetail

Customization

<https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-ann-rest-exceptions.html>



ProblemDetail



```
{
  "type": "https://problems-registry.smartbear.com/missing-body-property",
  "status": 400,
  "title": "Missing body property",
  "detail": "The request is missing an expected body property.",
  "code": "400-09",
  "instance": "/logs/regisrations/d24b2953-ce05-488e-bf31-67de50d3d085",
  "errors": [
    {
      "detail": "The body property {name} is required",
      "pointer": "/name"
    }
  ]
}
```

<https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/http/ProblemDetail.html>



ProblemDetail

Enable ProblemDetail for error by default

application.properties

```
spring.mvc.problemdetails.enabled=true
```



Example in ControllerAdvice

```
@RestControllerAdvice
class HelloControllerAdvice {

    @ExceptionHandler(InvalidInputException.class)
    public ProblemDetail handleInvalidInputException(
        InvalidInputException e, WebRequest request) {

        ProblemDetail problemDetail
            = ProblemDetail
                .forStatusAndDetail(HttpStatus.BAD_REQUEST, e.getMessage());
        problemDetail.setInstance(URI.create("demo-instance"));
        problemDetail.setTitle("demo");

        return problemDetail;
    }
}
```



Error handling

```
@Service
public class UserService {

    private AccountRepository accountRepository;

    @Autowired
    public UserService(AccountRepository accountRepository) {
        this.accountRepository = accountRepository;
    }

    public Account getAccount(int id) {
        Optional<Account> account = accountRepository.findById(id);
        if(account.isPresent()) {
            return account.get();
        }
        throw new MyAccountNotFoundException(
            String.format("Account id=[%d] not found", id));
    }
}
```



MyAccountNotFoundException

```
public class MyAccountNotFoundException  
    extends RuntimeException {
```

```
    public MyAccountNotFoundException(String message) {  
        super(message);  
    }
```

```
}
```



Response Status

404 = Not Found

Status	Description
400	Request body doesn't meet API spec
401	Authentication/Authorization fail
403	User can't perform the operation
404	Resource does not exist
405	Unsupported operation
500	Error on server



Handling error in Spring Boot

```
@RestControllerAdvice  
public class AccountControllerHandler {
```

1

```
    @ExceptionHandler(MyAccountNotFoundException.class)  
    public ResponseEntity<ExceptionResponse> accountNotFound(  
        MyAccountNotFoundException exception) {  
  
        ExceptionResponse response =  
            new ExceptionResponse(exception.getMessage(),  
                                   "More detail");  
  
        return new ResponseEntity<ExceptionResponse>(response,  
                                                       HttpStatus.NOT_FOUND);  
    }  
}
```



Handling error in Spring Boot

```
@RestControllerAdvice
public class AccountControllerHandler {

    @ExceptionHandler(MyAccountNotFoundException.class)
    public ResponseEntity<ExceptionResponse> accountNotFound(
        MyAccountNotFoundException exception) {

        ExceptionResponse response =
            new ExceptionResponse(exception.getMessage(),
                                "More detail");

        return new ResponseEntity<ExceptionResponse>(response,
                                                    HttpStatus.NOT_FOUND);
    }
}
```

2



ExceptionResponse

Response format of error

```
public class ExceptionResponse{  
  
    private Date timestamp = new Date();  
    private String message;  
    private String detail;  
  
    public ExceptionResponse(String message, String detail) {  
        this.message = message;  
        this.detail = detail;  
    }  
}
```



Result of API

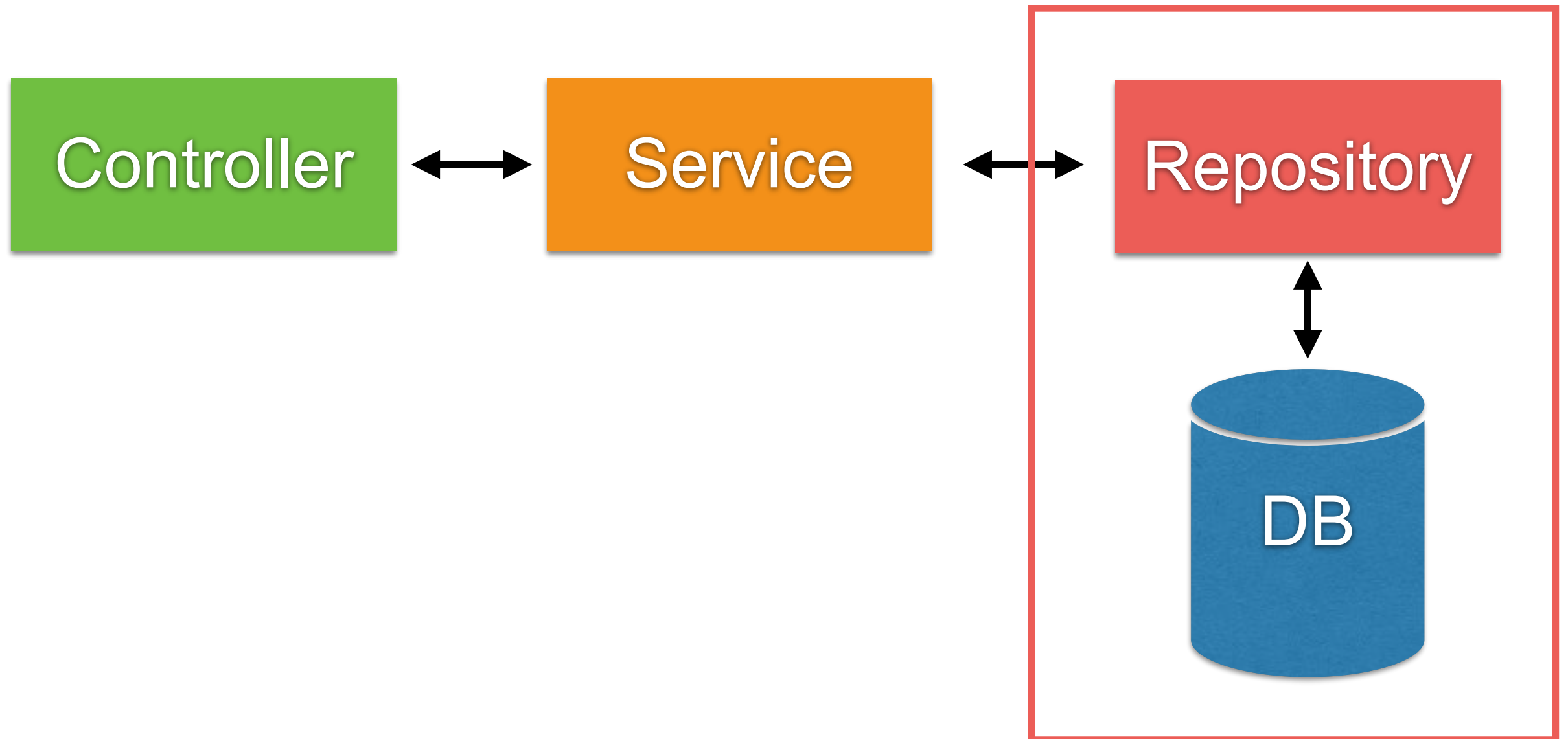
```
localhost:8888/account/2  
{  
  timestamp: "2018-09-15T15:25:44.776+0000",  
  message: "Account id=[2] not found",  
  detail: "More detail"  
}
```



Working with Database



Repository Layer in Spring



Basic of JDBC

```
// 1. Load jdbc driver
Class.forName("postgresql");

// 2. Create connection
Connection connection = DriverManager.getConnection("", "", "");

// 3. Prepared Statement
String sql = "SELECT * FROM TABLE WHERE name=?";
PreparedStatement pstmt = connection.prepareStatement(sql);

// 4. Query
ResultSet resultSet = pstmt.executeQuery();
while(resultSet.next()) {

}

// 5. Release resource
if(resultSet != null) {
    resultSet.close();
    resultSet = null;
}
```



Framework !!

```
// 1. Load jdbc driver
Class.forName("postgresql");

// 2. Create connection
Connection connection = DriverManager.getConnection("", "", "");
```

Manage by Framework

```
// 3. Prepared Statement
String sql = "SELECT * FROM TABLE WHERE name=?";
PreparedStatement pstmt = connection.prepareStatement(sql);

// 4. Query
ResultSet resultSet = pstmt.executeQuery();
while(resultSet.next()) {

}
```

```
// 5. Release resource
if(resultSet != null) {
    resultSet.close();
    resultSet = null;
}
```

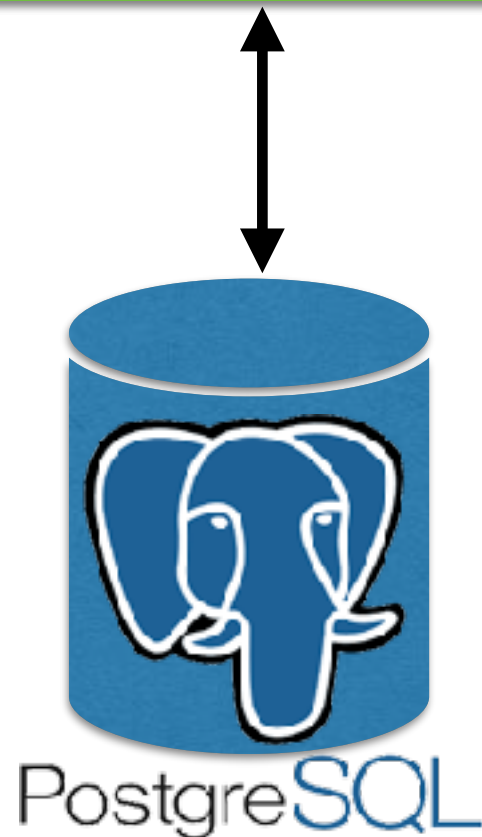
Manage by Framework



Working with Database ?

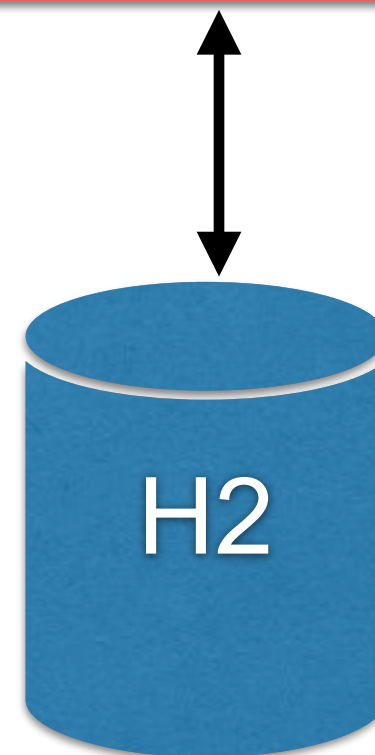
Production

Application



Testing

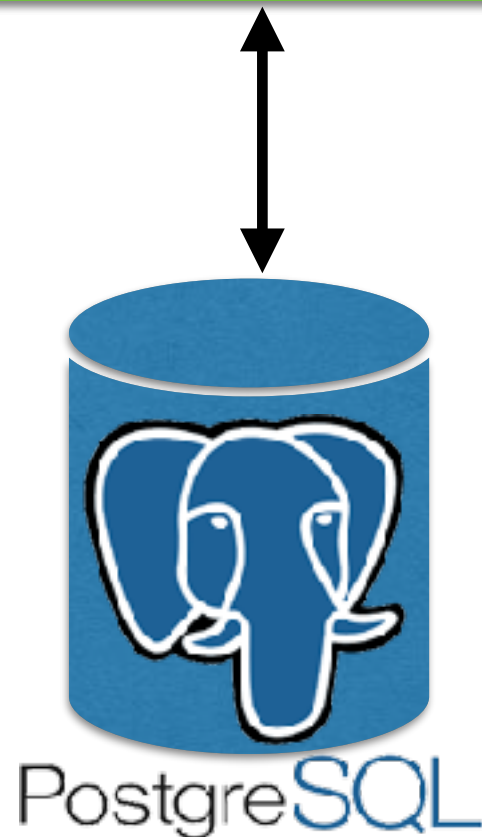
Application



Working with Database ?

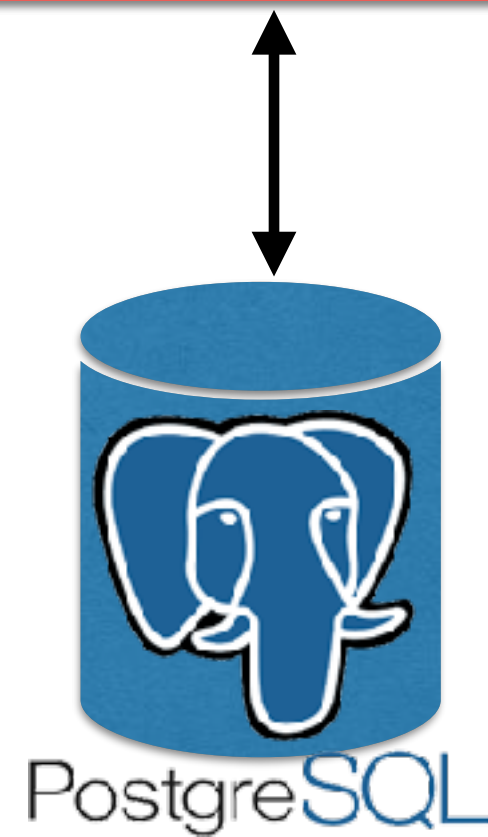
Production

Application



Testing

Application



Working with repository

We're using Spring Data



<https://spring.io/projects/spring-data>

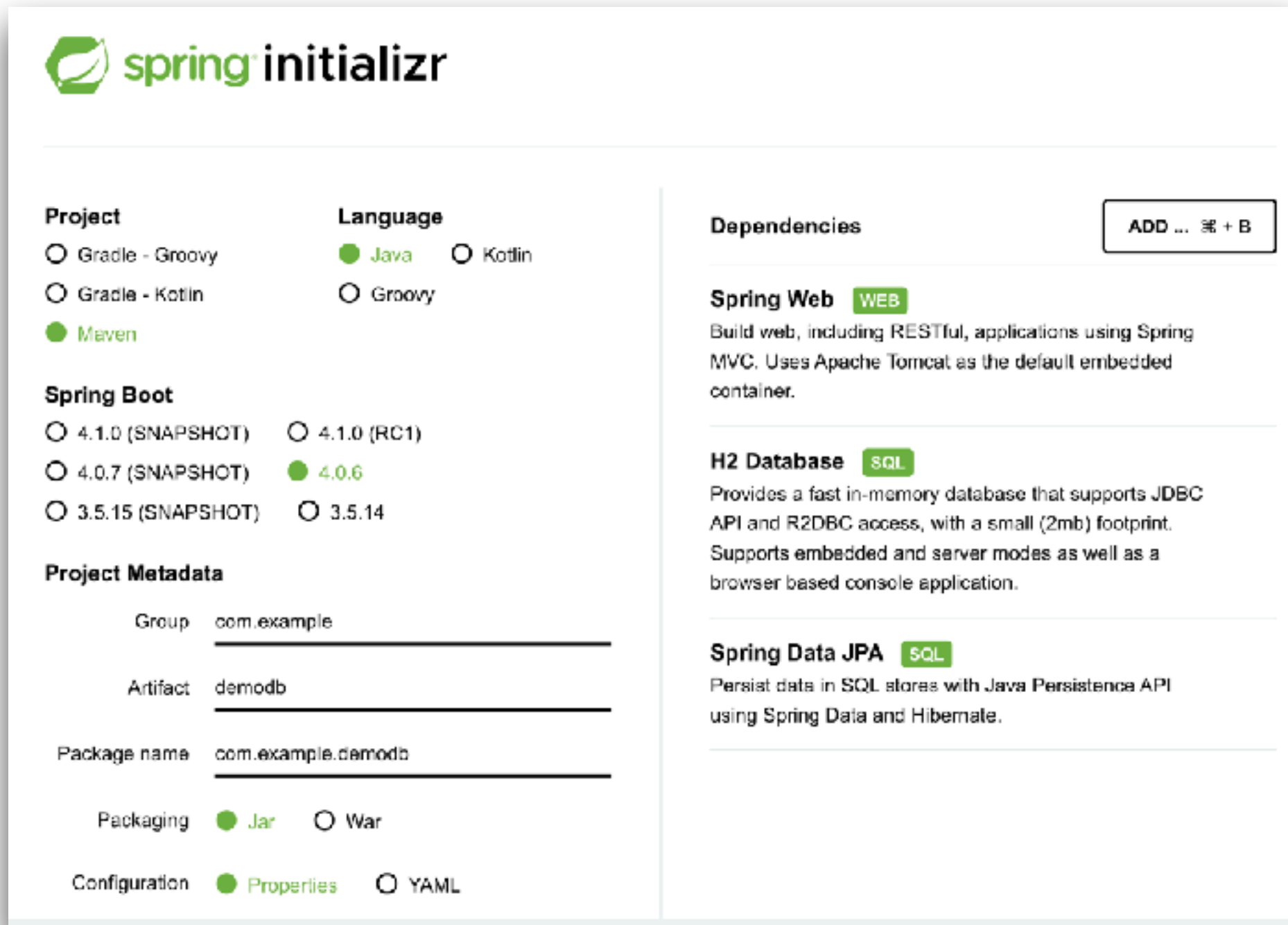


Working with Database

Manage transactions with @Transactional
Manage multiple datasources



Create Project



The image shows the Spring Initializr web interface for creating a new project. It is divided into several sections: Project, Language, Spring Boot, Project Metadata, Dependencies, and a list of selected dependencies.

Project

- ☐ Gradle - Groovy
- ☐ Gradle - Kotlin
- ☒ Maven

Language

- ☒ Java
- ☐ Kotlin
- ☐ Groovy

Spring Boot

- ☐ 4.1.0 (SNAPSHOT)
- ☐ 4.1.0 (RC1)
- ☐ 4.0.7 (SNAPSHOT)
- ☒ 4.0.6
- ☐ 3.5.15 (SNAPSHOT)
- ☐ 3.5.14

Project Metadata

Group:

Artifact:

Package name:

Packaging: ☒ Jar ☐ War

Configuration: ☒ Properties ☐ YAML

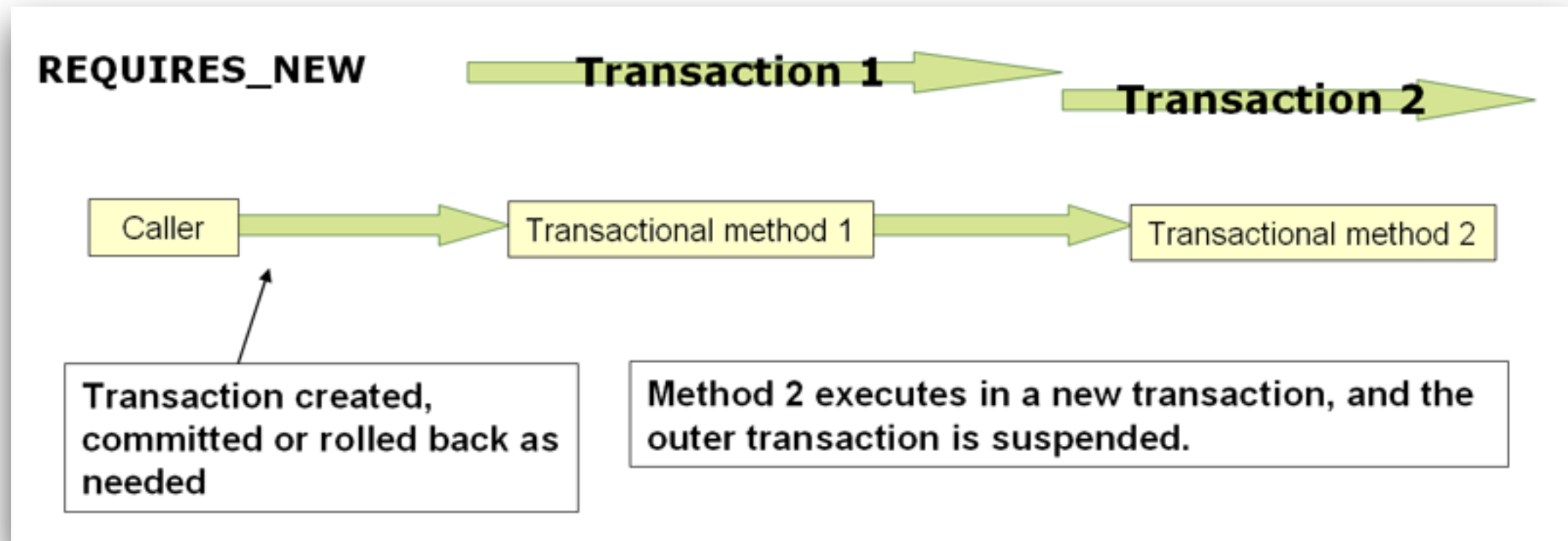
Dependencies ADD ... ⌵ + B

- Spring Web** WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
- H2 Database** SQL
Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.
- Spring Data JPA** SQL
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

<https://start.spring.io/>



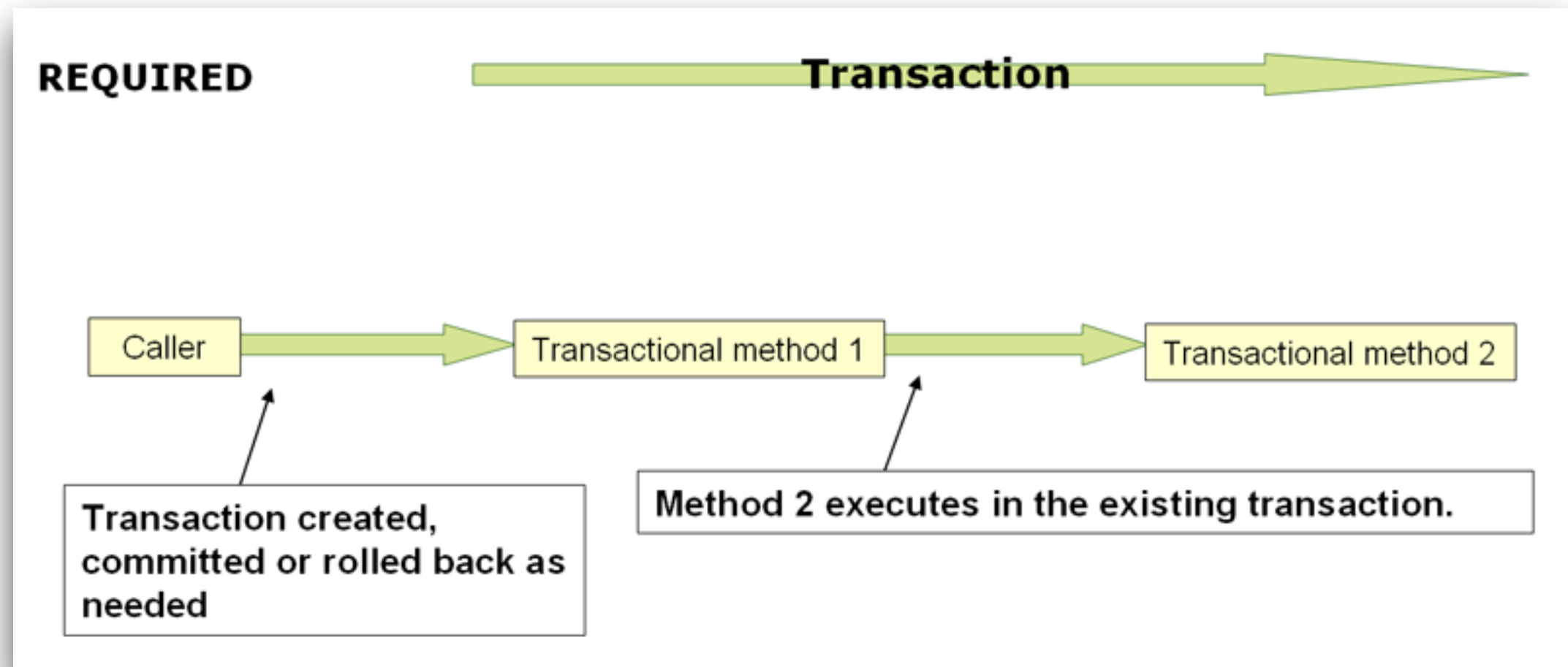
Transaction Propagation (1)



<https://start.spring.io/>



Transaction Propagation (2)



<https://start.spring.io/>



Q/A

